

우리금융그룹 AI/데이터 전문가 과정

DB for AI

2026. 05.



Prof. Sang-goo Lee

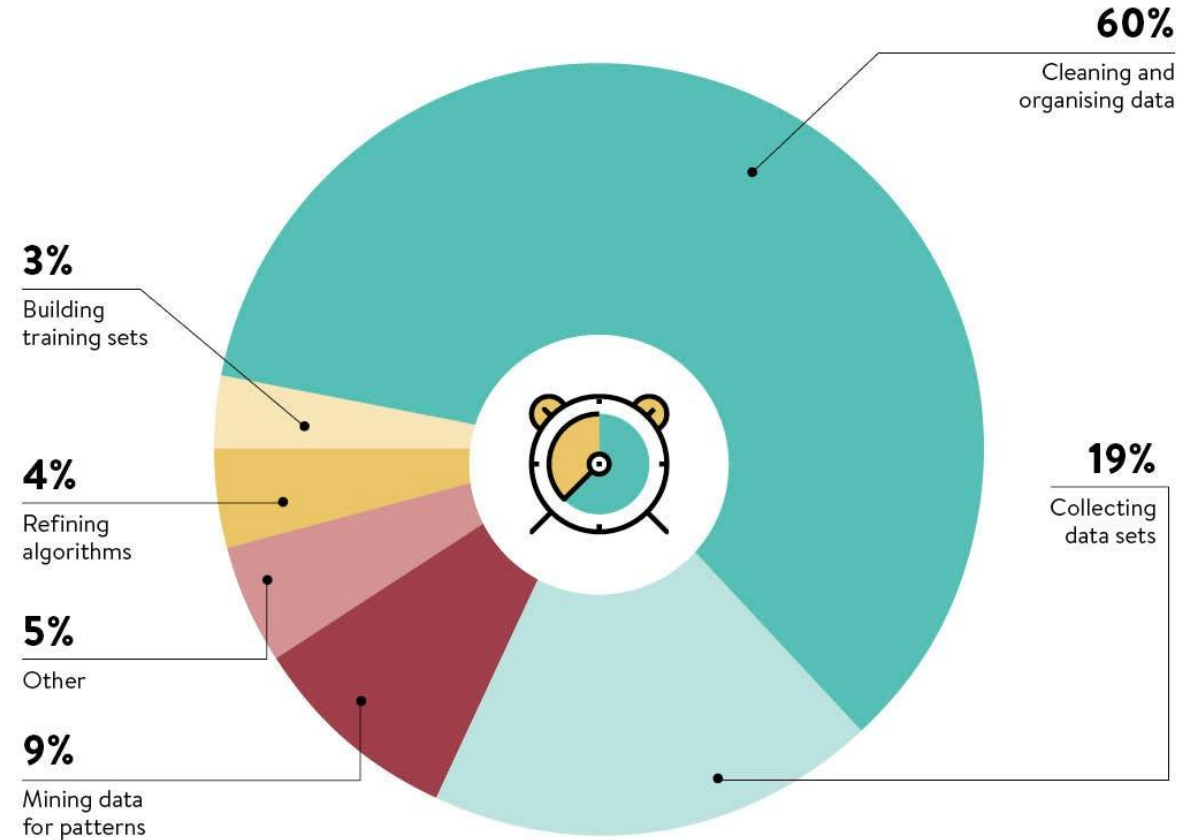
Intelligent Data Systems Laboratory
Computer Science & Engineering
Seoul National University

IntelliSys
A.I. DRIVEN INNOVATIONS

*'AI is here!
But needs a lot of engineering!'*



WHAT DATA SCIENTISTS SPEND THE MOST TIME DOING



DB for AI

I. 개요

1. DB 역사

II. 관계형 DB & SQL

2. 관계형 DB
3. SQL – 기본
4. SQL – 집계함수
5. SQL - 조인
6. 데이터 분석

III. DB & AI

7. DB for ML – Feature Store
8. LLM for DB – Text2SQL
9. DB for LLM

IV. 실습

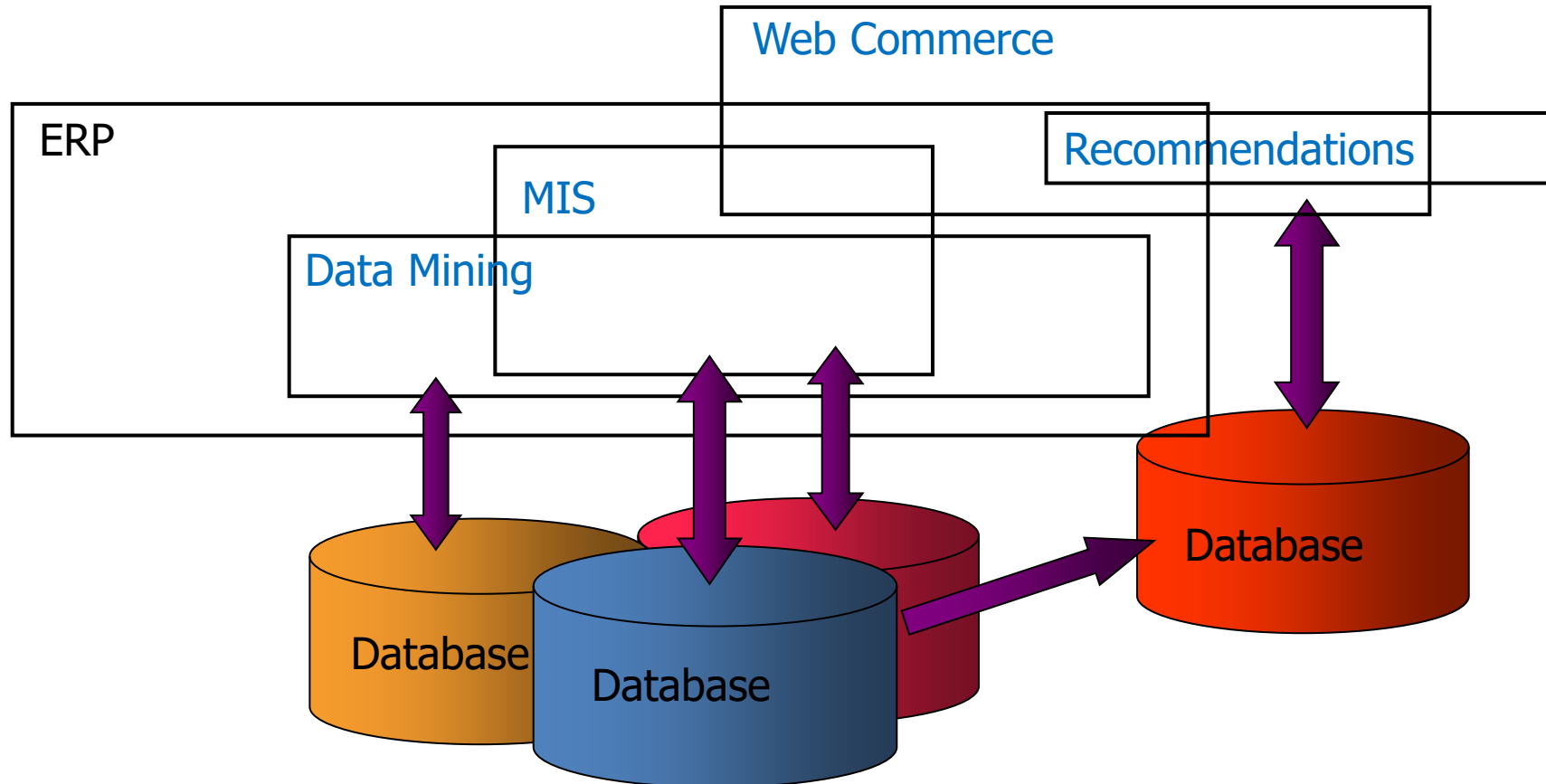
- A. SQL 기본 - Stepik
- B. 분석 실습 – Jupyter Notebook

1. DB 역사



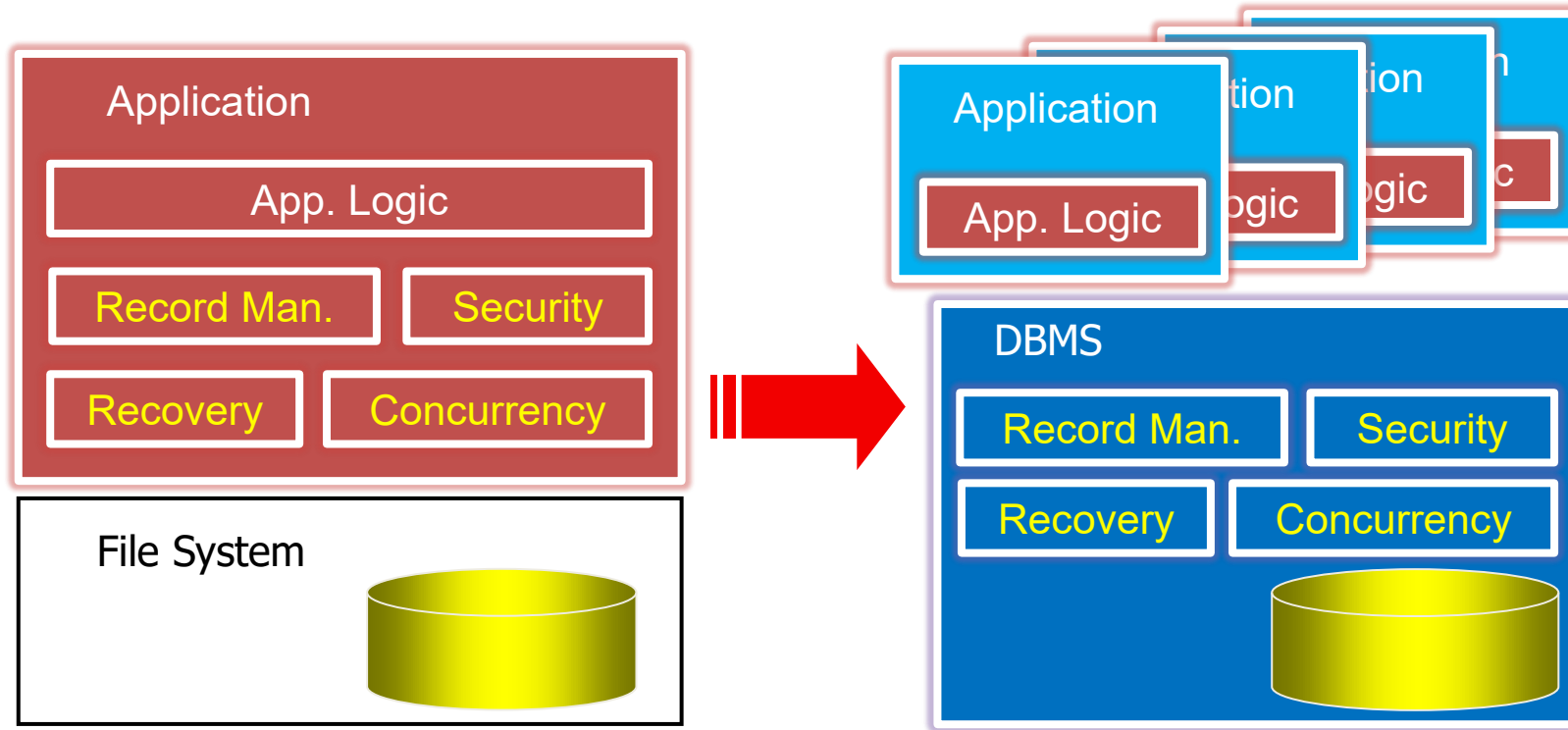
Computing = Data Processing

- 대부분(모든?) 전산 시스템은 어떤 형태로든 데이터베이스를 활용하게 된다



기본: File System

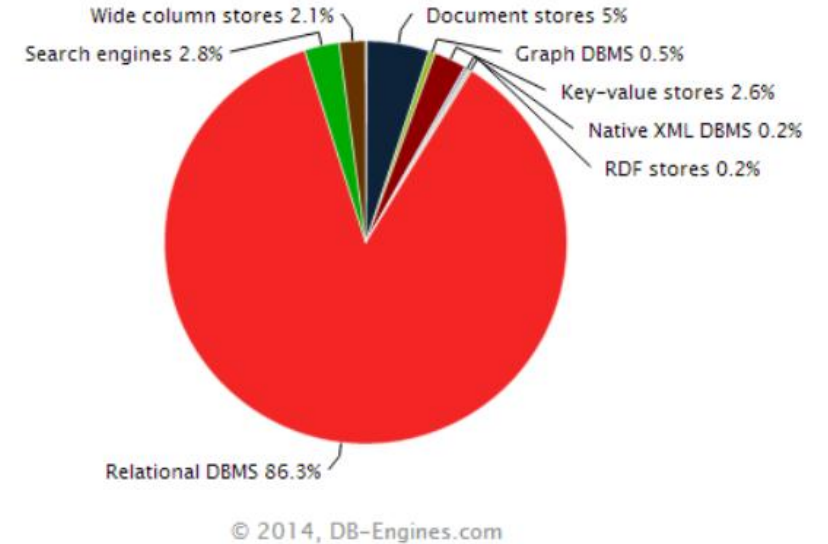
- File System (FS)
 - OS의 핵심 하부 모듈
 - 프로그램(코드), 데이터, 문서, 영화 등 저장 관리
 - (in disk)



- Database Management System (DBMS)
 - 데이터의 저장, 관리, 처리 기능을 제공하는 소프트웨어 집합 (시스템)

[Tipping Point 1] Data Base – Relational (관계형)

- 단순하고 직관적인 구조 (모델)
- 쉽고 강력한 DB 언어 (SQL)
- 자동최적화를 통한 처리 속도
- 안정된 업무처리 지원 (transaction support)



ID	name	dept_name	salary
22222	Einstein	Physics	95000
12121	Wu	Finance	90000
32343	El Said	History	60000
45565	Katz	Comp. Sci.	75000
98345	Kim	Elec. Eng.	80000
76766	Crick	Biology	72000
10101	Srinivasan	Comp. Sci.	65000
58583	Califieri	History	62000
83821	Brandt	Comp. Sci.	92000
15151	Mozart	Music	40000
33456	Gold	Physics	87000
76543	Singh	Finance	80000

ID	course_id	sec_id	semester	year
10101	CS-101	1	Fall	2009
10101	CS-315	1	Spring	2010
10101	CS-347	1	Fall	2009
12121	FIN-201	1	Spring	2010
15151	MU-199	1	Spring	2010
22222	PHY-101	1	Fall	2009
32343	HIS-351	1	Spring	2010
45565	CS-101	1	Spring	2010
45565	CS-319	1	Spring	2010
76766	BIO-101	1	Summer	2009
76766	BIO-301	1	Summer	2010
83821	CS-190	1	Spring	2009
83821	CS-190	2	Spring	2009
83821	CS-319	2	Spring	2010
98345	EE-181	1	Spring	2009

dept_name	building	budget
Biology	Watson	90000
Comp. Sci.	Taylor	100000
Elec. Eng.	Taylor	85000
Finance	Painter	120000
History	Painter	50000
Music	Packard	80000
Physics	Watson	70000

증거기반 의사결정 (Evidence Based Decision Making)

- 데이터 기반의 통찰력(Insights) & 예지력(foresights)

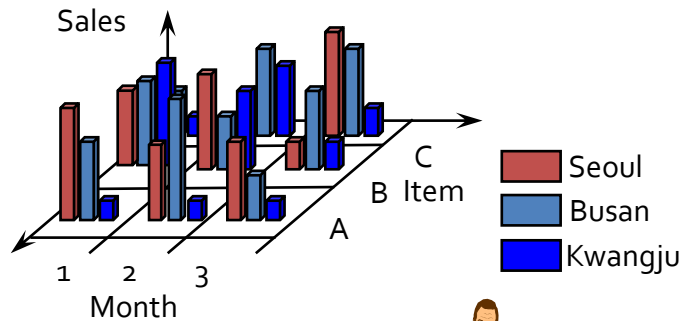
*"It is a capital mistake to theorize before one has data.
Insensibly one begins to twist facts to suit theories,
instead of theories to suit facts."*



- The Adventures of Sherlock Holmes, A. Conan Doyle

- But ...

다차원적 분석 관점
(multidimensional/analytic view)



의사결정 임원

평면적/실행적 관점
(tabular/transactional view)

ID	name	dept_name	salary
22222	Einstein	Physics	95000
12121	Wu	Finance	90000
32343	El Said	History	60000
45565	Katz		
98345	Kim	BIO-101	
76766	Crick	BIO-309	
10101	Scriniv	CS-101	
58583	Califia	CS-190	
83821	Brand	CS-315	
15151	Moza	CS-319	
33456	Gold	CS-347	
76543	Singh	EE-181	

dept_name	building	budget
Biology	Watson	90000
Comp. Sci.	Taylor	100000
Elec. Eng.	Taylor	85000
Finance	Painter	120000
History	Painter	50000

course_id	prereq_id	prereq_title	production
BIO-301	BIO-101		
BIO-309	BIO-101		
CS-190	CS-101		
CS-315	CS-101		
CS-319	CS-101		
CS-347	CS-101		
EE-181	PHY-101		

dept_name	building	budget
Biology	Watson	90000
Comp. Sci.	Taylor	100000
Elec. Eng.	Taylor	85000
Finance	Painter	120000
History	Painter	50000

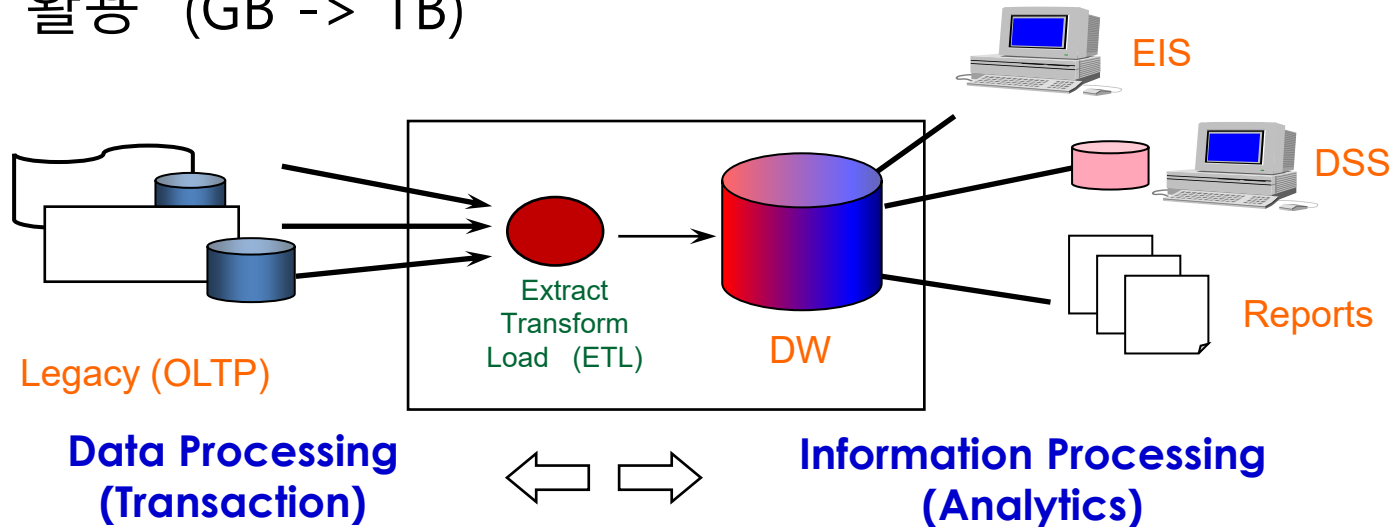


IT 부서

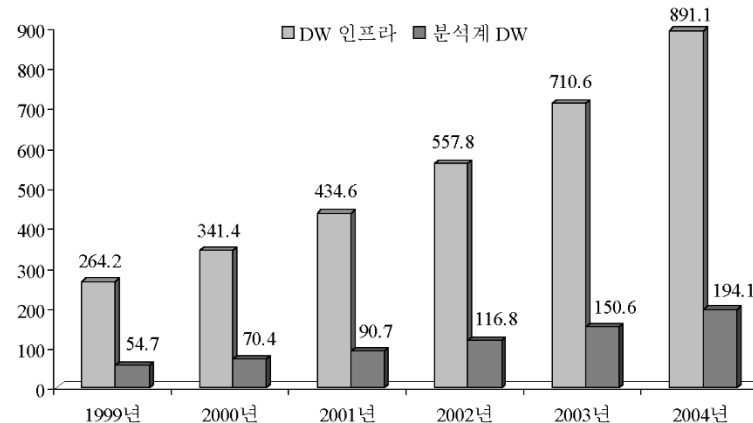
비효율적
의사결정 사이클

[Tipping Point 2] Data Warehouse

- 업무시스템(transaction system)으로부터 쌓이는 데이터를 한 곳에 모아
- 분석적 작업에 활용 (GB -> TB)

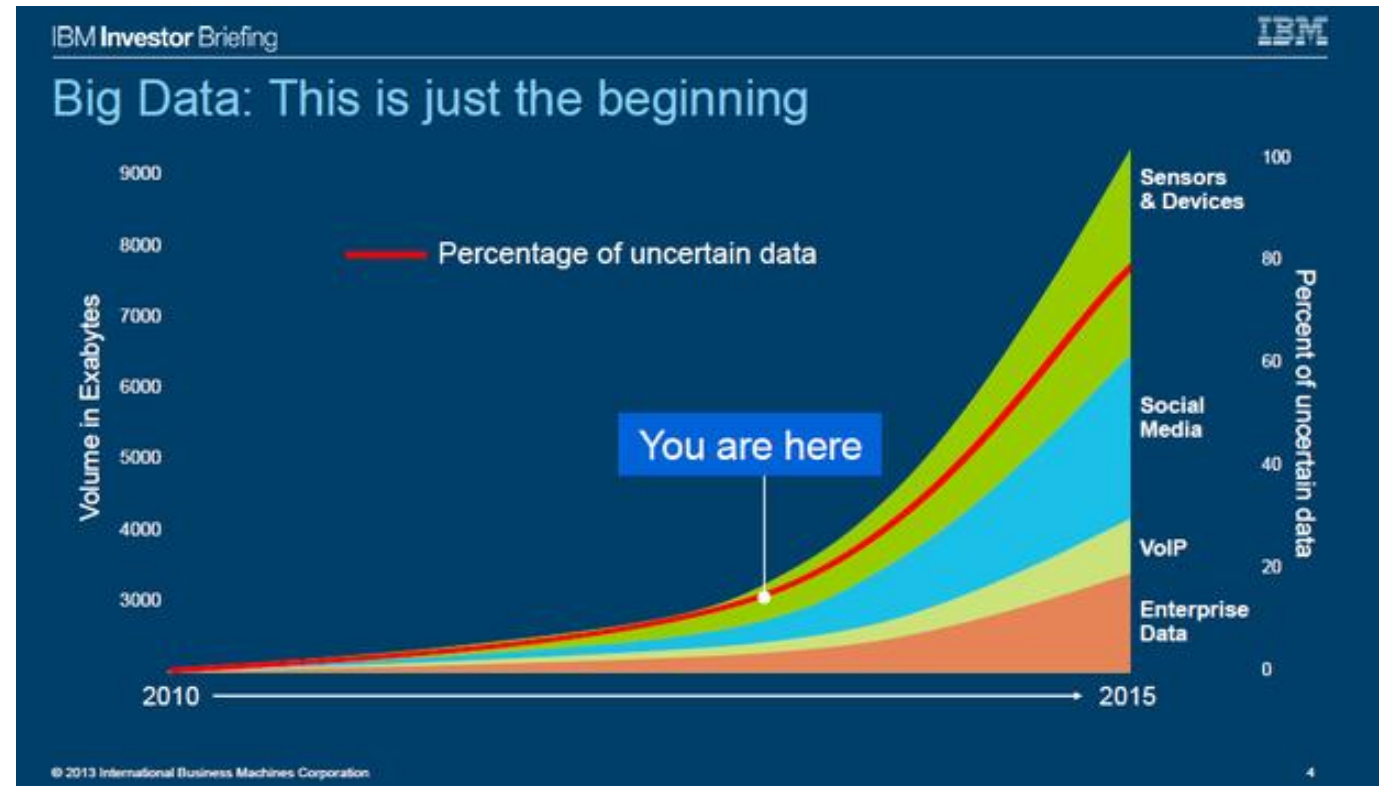


=> 데이터 분석의 전성기



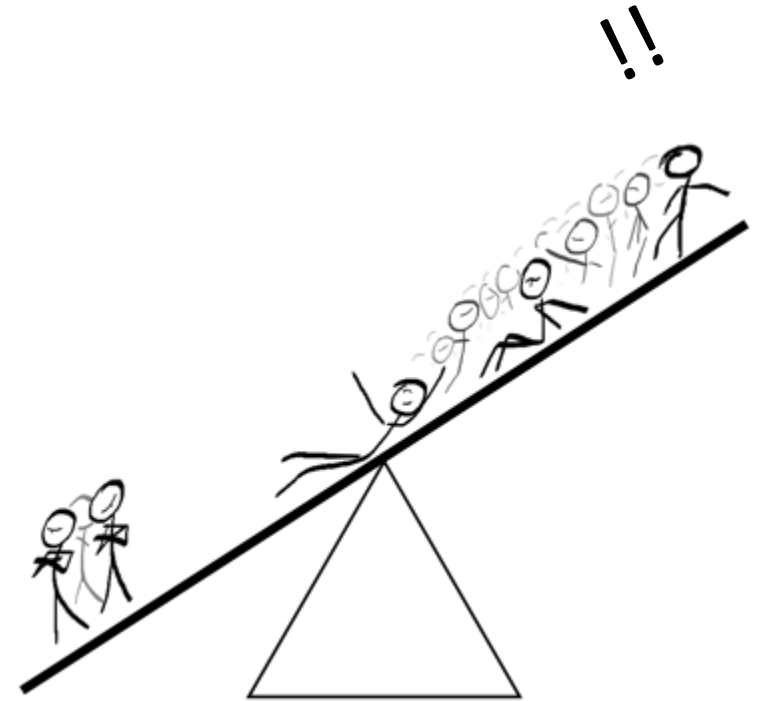
데이터 폭발 (Data Explosion)

- DW 는 빙산의 일각 – 기업 데이터 증가보다 더 빠르게 팽창
- 1.5년마다 2배로 증가!
- Data growth 요인
 - “SW is eating the world”
– 모든 곳의 전산화/정보화
 - Mobile & social networks
 - Sensors & smart devices



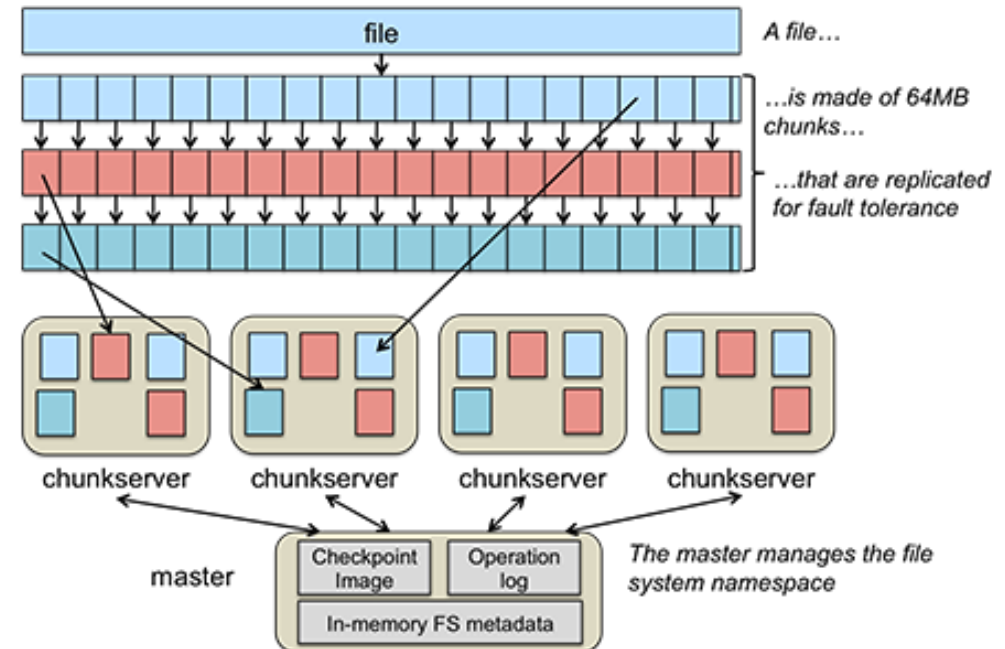
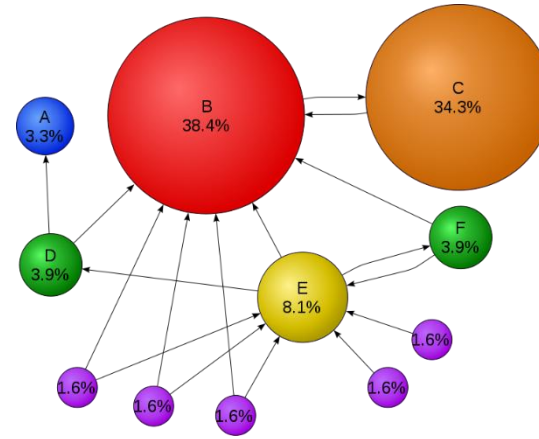
[Tipping Point 3] Big Data

- **Ubiquitous** – 모든 분야에 일어나고 있는 현상
 - 생산, 유통, 의료, 공공, 문화, 언론, 역사, ...
 - 정보화/자동화, 모바일, 소셜, 센서!!
 - Impact 있는 사례
- **Feasible** – 효과적으로 대응할 수 있는 환경
 - 풍부한 데이터
 - 강력한 컴퓨팅 자원
 - 효과적인 분석 기술
- **Virtuous Cycle** – 데이터 기반 해결책의 가치 인정
 - 분위기 전환 – more and more success stories
 - Data가 핵심 자산이라는 인식 확산
 - 연계/통합/융합으로 새로운 기회 발굴



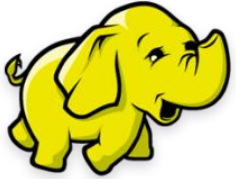
Web Scale Computing

- Different data,
- Different operations,
- Different scale!



Big Data Systems

- Hadoop



- Apache Open Source SW
- MapReduce 기반의 대량 데이터 분산처리 framework
- Yahoo!에서 시작/지원 (2006)

- NoSQL

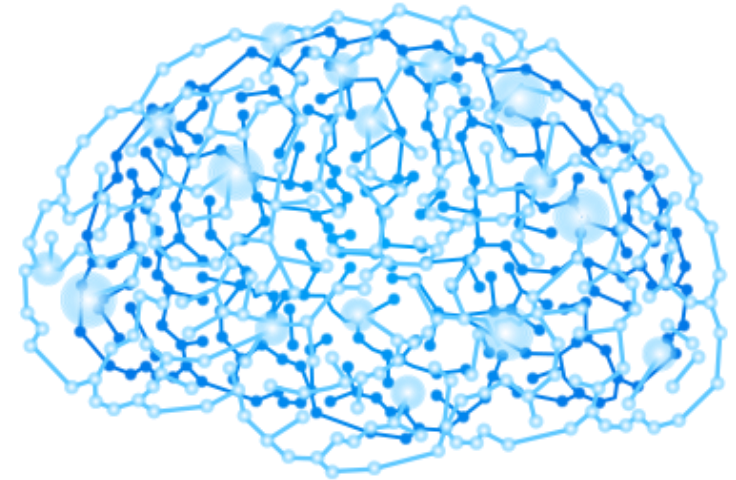


- Not Only (?) SQL
- 단순한 데이터 모델: Key-Value store
- 단순한 질의: get(), put()
- 단순한 트랜잭션 모델: BASE – Basically Available, Soft state, Eventual consistency

[Tipping Point 4] AI – Machine Learning

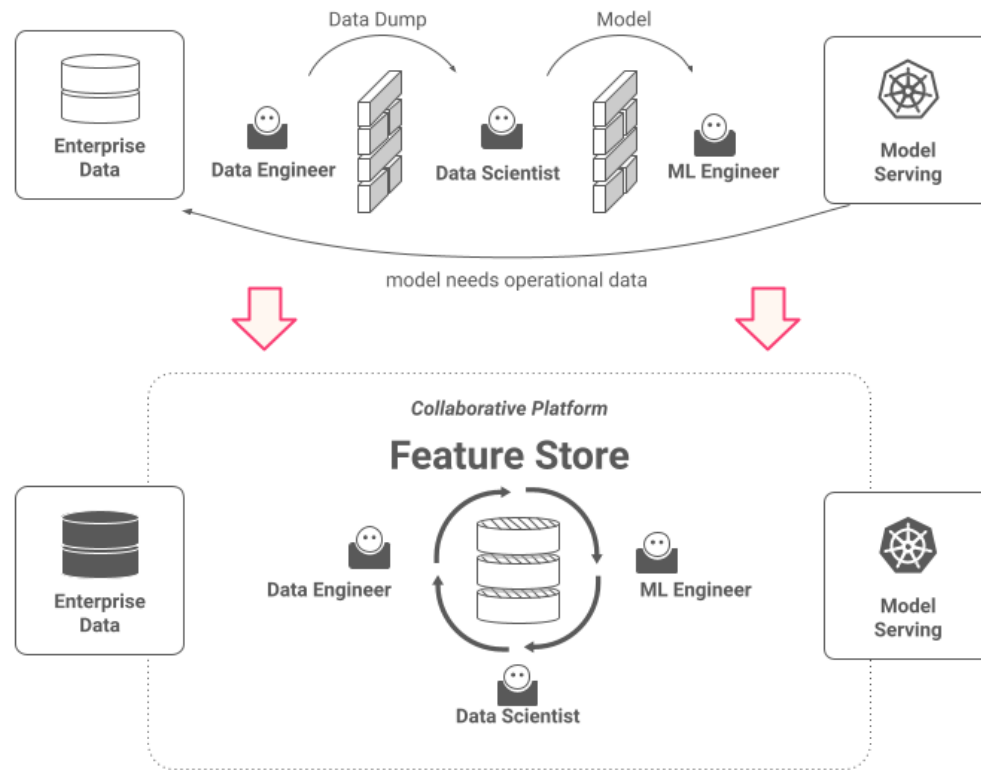
전통적 인공지능 분야 문제 해결

- 컴퓨터 비전 (Computer Vision)
 - 자연어 처리 (Natural Language Processing)
 - Google Translate, Narrative Science
 - 자율 주행
 - Generative AI
 - GPT, Sora, Veo
- 기계학습은 *컴퓨터 입장에서*의 '증거기반 의사결정'



Feature Store

- 기계학습에 사용되는 데이터의 체계적인 관리
- 학습단계와 운영단계 지원 (MLOps)



A "feature" refers to the entire column in the dataset

Transaction_id	in_foreign_country	size_compared_to_avg_transaction	fraud?
7485	False	0.8x	False
46854	True	21.2x	True
3521	True	1.1x	False

A "feature value" refers to a single value of a feature column

[Tipping Point 5] 생성형 AI

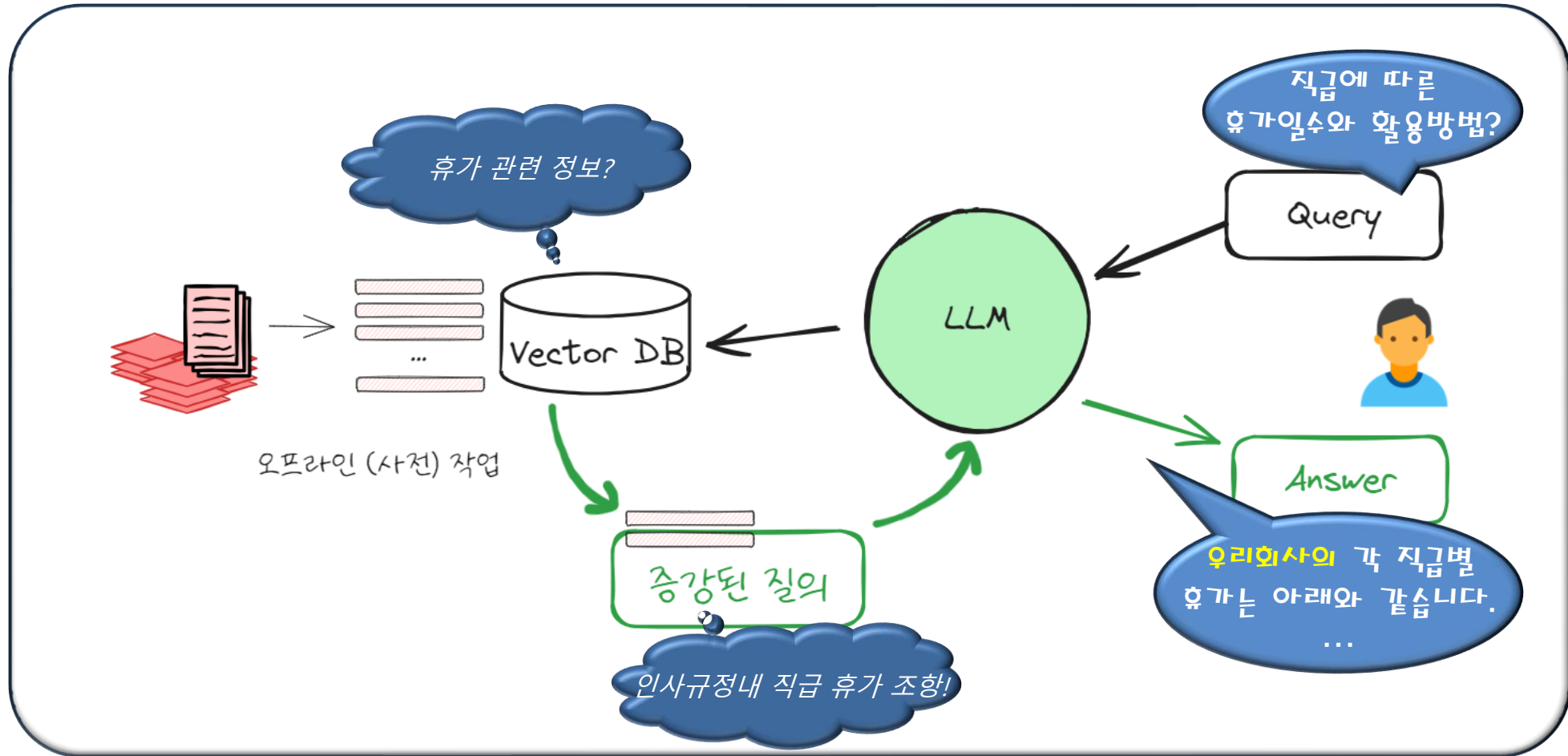


- ✓ 지식 단절성
- ✓ 1회성 질의응답
- ✓ 프라이버시와 보안

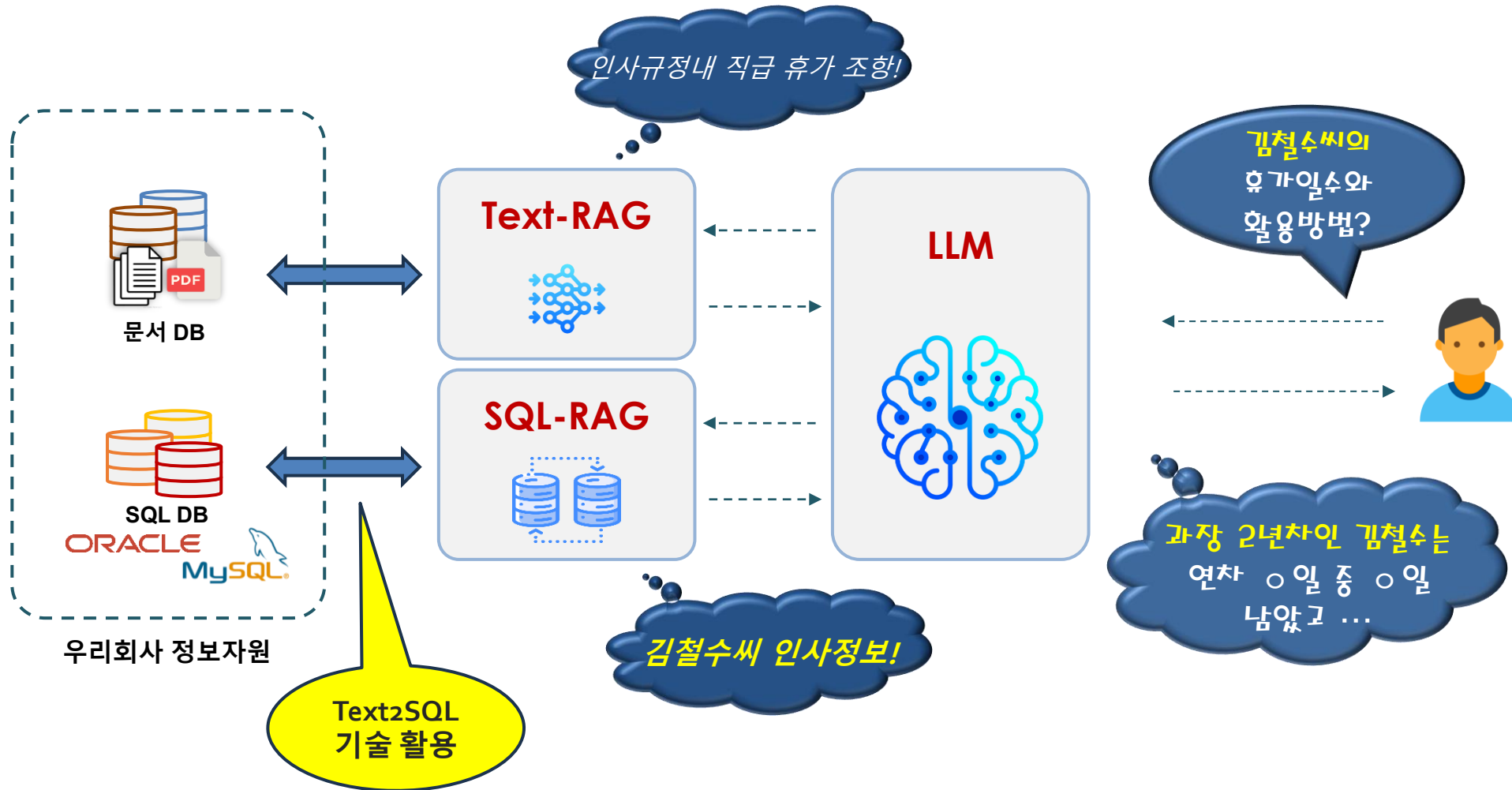


RAG – Retrieval Augmented Generation (검색증강생성)

우리회사 정보와 연결!



RAG++



Text2SQL

A지점에
S카드 여신잔액
있는 고객 정보
알려줘!

AI Spreadsheets

HomeHistory로그아웃

다른 분석을 요청하고 싶으시면, "새 채팅"을 눌러주세요!

+ 새 채팅

2024년 카드 상품별 여신 잔액 보여줘

2024년 카드 상품별 여신 잔액 확인 요청
2024년 카드 상품 중 'S2 캐시체크카드'의 여신 잔액이 815,026,604원으로 가장 높습니다.
다음으로 '핀카드'가 611,394,596원으로 두 번째로 높은 여신 잔액을 기록했습니다.

핀카드 여신잔액이 가장 높은 지점은?

2024년 핀카드 여신 잔액이 가장 높은 지점 확인 요청
2024년 핀카드 여신 잔액이 가장 높은 지점은 삼이로길지점이며, 여신 잔액은 132,264,430원입니다.

삼이로길지점 핀카드 여신 잔액이 있는 고객 정보 알려줘

2024년 핀카드 여신 잔액이 가장 높은 지점과 고객 정보 요청
삼이로길지점의 핀카드 여신 잔액이 가장 높은 고객은 오호기입니다.
오호기의 여신 잔액은 94,756,609원이며, 고객번호는 5,593,475입니다.

오호기 고객님의 여신잔액이 있는 다른 상품 있어?

카드/대출

3 columns 6 rows

오호기 고객님의 여신잔액이 있는 다른 상품 있어?

삼이로길지점 핀카드 여신 잔액이 있는 고객 정보 알려줘

핀카드 여신잔액이 가장 높은 지점은?

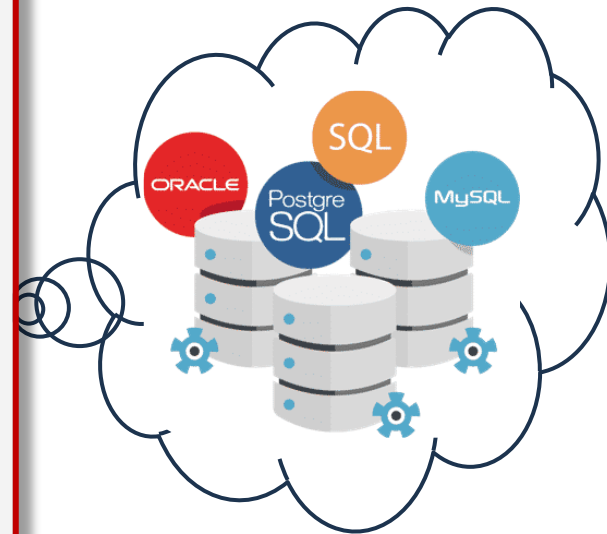
2024년 카드 상품별 여신 잔액 보여줘

전체보기

고객명	상품명	여신잔액
오호기	핀카드	96,538,206
오호기	핀카드	94,756,609
오호기	비대면예적금담보대출	87,227,050
오호기	심심아수혜이체크카드	41,646,834
오호기	헤이(Hey)플러스대출	76,101,094
오호기	Sh모두행복계기업적금	81,936,935

« < 1 > »

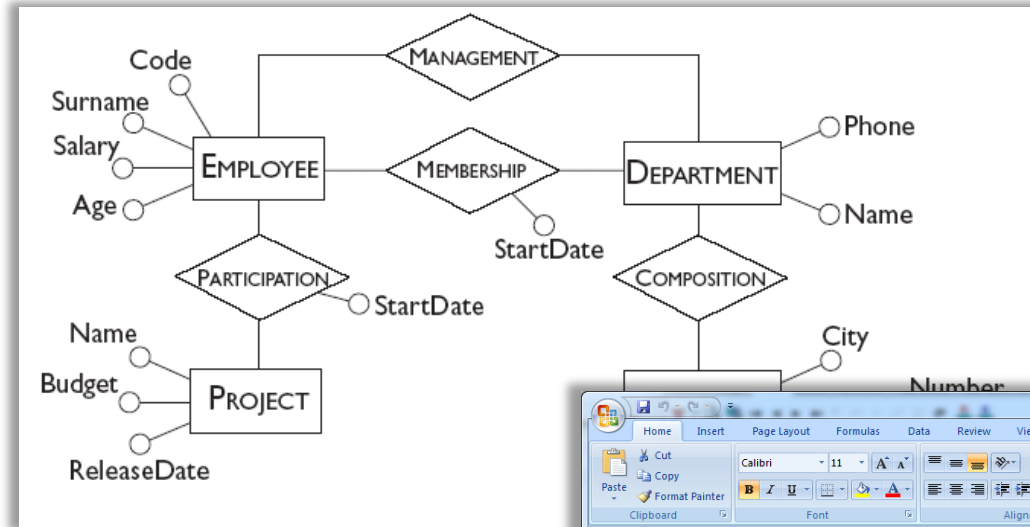
Copyright © 2025 IntelliSys All Rights Reserved.



2. 관계형 데이터베이스 (Relational DB)



데이터 모델 (Data Model)



Keys	Column families		
a	colA:value1	colFoo:a value	fram:zilk
b	colA:value1	colB:a value	♠: chesspiece
bb	colA:value1	colB:	♫: 🎵
			colFoo:a value

Sheet1 - Microsoft Excel

EMP_ID	JOB_ID	DPT_NAME	EMP_NAME
77514063	AC_ACCOUNT	LI	KATHY ABELLERA
21328077	AC_ACCOUNT	LI	JEREMIAH DINETZ
63114872	AC_ACCOUNT	LI	WENDALEE BURNETT
22447298	AC_ACCOUNT	LI	MISSY KOWAL
42823120	AC_ACCOUNT	LI	JEHU POWNELL
73022131	AC_ACCOUNT	LI	BAZE WALTON
30206729	AC_ACCOUNT	LI	DEE TOOF
68388930	AC_ACCOUNT	LI	MURRAY WALEN
52994681	AC_ACCOUNT	LI	TODD CORNELL
85531859	AC_ACCOUNT	LI	CRAIG HARRINGTON
99625806	AC_ACCOUNT	LI	DALIA DEROSIER
85952542	AC_ACCOUNT	LI	JERAMY CAMPISE
90514846	AC_ACCOUNT	LI	ERV BRODSHO
10293371	AC_ACCOUNT	LI	RODGER WAKEFIELD
93331872	AC_ACCOUNT	LI	GEODIE ROEHL
38996915	AC_ACCOUNT	LI	BRUCE BULAGA
95488542	AC_ACCOUNT	LI	EDWARD JOSEPH
43800025	AC_ACCOUNT	LI	BRIAN FURNO
22735685	AC_ACCOUNT	LI	VERMELL DRESSER
11249307	AC_ACCOUNT	LI	EILEEN GILBERTSON
30157047	AC_ACCOUNT	LI	ANGELIA HOWETH
62528105	AC_ACCOUNT	LI	KASEY STRAMANDINOLI
64880465	AC_ACCOUNT	LI	GRAIG DOWNER
68221632	AC_ACCOUNT	LI	OCTAVIA LAMOREAUX
87701077	AC_ACCOUNT	LI	AUTUMN BOLTON
33060000	AC_ACCOUNT	LI	DEAN DOUGHER

관계형 데이터 모델

ID	name	dept_name	salary
22222	Einstein	Physics	95000
12121	Wu	Finance	90000
32343	El Said	History	60000
45565	Katz	Comp. Sci.	75000
98345	Kim	Elec. Eng.	80000
76766	Crick	Biology	72000
10101	Srinivasan	Comp. Sci.	65000
58583	Califiori	History	62000
83821	Brandt	Comp. Sci.	92000
15151	Mozart	Music	40000
33456	Gold	Physics	87000
76543	Singh	Finance	80000

dept_name	building	budget
Biology	Watson	90000
Comp. Sci.	Taylor	100000
Elec. Eng.	Taylor	85000
Finance	Painter	120000
History	Painter	90000

course_id	title	dept_name	credits
BIO-101	Intro. to Biology	Biology	4
BIO-301	Genetics	Biology	4
BIO-399	Computational Biology	Biology	4
CS-101	Intro. to Computer Science	Comp. Sci.	3
CS-190	Game Design	Comp. Sci.	3
CS-315	Robotics	Comp. Sci.	3
CS-319	Image Processing	Comp. Sci.	3
CS-347	Database System Concepts	Comp. Sci.	3
EE-181	Intro. to Digital Systems	Elec. Eng.	3
FIN-201	Investment Banking	Finance	3
HIS-351	World History	History	3

course_id	prereq_id	production
BIO-301	BIO-101	1
BIO-399	BIO-101	1
CS-190	CS-101	1
CS-315	CS-101	1
CS-319	CS-101	1
CS-347	CS-101	1
EE-181	PHY-101	1

속성, 컬럼, 필드
(attributes, columns, fields)

instructor
테이블, 릴레이션, 관계
(table, relation)

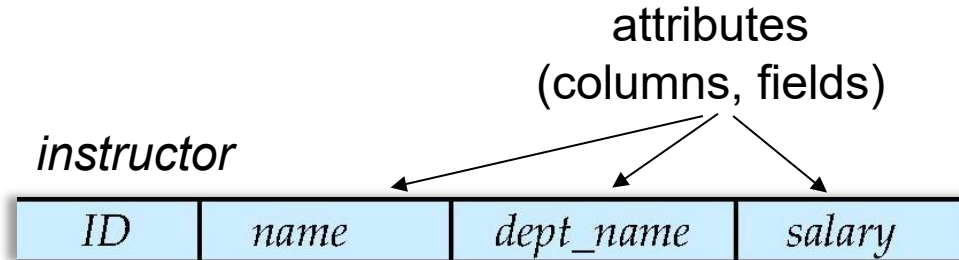
ID	name	dept_name	salary
10101	Srinivasan	Comp. Sci.	65000
12121	Wu	Finance	90000
15151	Mozart	Music	40000
22222	Einstein	Physics	95000
32343	El Said	History	60000
33456	Gold	Physics	87000
45565	Katz	Comp. Sci.	75000
58583	Califiori	History	62000
76543	Singh	Finance	80000
76766	Crick	Biology	72000
83821	Brandt	Comp. Sci.	92000
98345	Kim	Elec. Eng.	80000

레코드, 행, 튜플
(records, rows, tuples)
- 실세계의 하나의 개체/이벤트 표현

스키마 & 인스턴스

■ Schema

- 테이블이 어떤 속성으로 구성되는가
- 테이블의 골격



■ Instance

- 테이블이 어떤 데이터 값들로 채워져 있나
- 테이블의 현재 상태

records
(rows, tuples)

<i>ID</i>	<i>name</i>	<i>dept_name</i>	<i>salary</i>
10101	Srinivasan	Comp. Sci.	65000
12121	Wu	Finance	90000
15151	Mozart	Music	40000
22222	Einstein	Physics	95000
32343	El Said	History	60000
33456	Gold	Physics	87000
45565	Katz	Comp. Sci.	75000

instructor

관계형 데이터베이스 (Relational DB)

- (관계형) DB = 테이블들의 집합
- 대상 정보 전체를 담아야

instructor (교수)

ID	name	dept_name	salary
22222	Einstein	Physics	95000
12121	Wu	Finance	90000
32343	El Said	History	60000
45565	Katz	Comp. Sci.	75000
98345	Kim	Elec. Eng.	80000
76766	Crick	Biology	72000
10101	Srinivasan	Comp. Sci.	65000
58583	Califieri	History	62000
83821	Brandt	Comp. Sci.	92000
	Mozart	Music	40000
	Gold	Physics	87000
	Singh	Finance	80000

course (교과목)

dept_name	building	budget	course_id	title	dept_name	credits
Biology	Watson	90000	BIO-101	Intro. to Biology	Biology	4
Comp. Sci.	Taylor	100000	BIO-301	Genetics	Biology	4
Elec. Eng.	Taylor	85000	BIO-399	Computational Biology	Biology	3
Finance	Painter	120000	CS-101	Intro. to Com		
History	Painter	50000	CS-190	Game Design		
Music	Packard	80000	CS-315	Robotics		
Physics	Watson	70000	CS-319	Image Proces		
			CS-347	Database Sys		
			EE-181	Intro. to Digi		
			FIN-201	Investment B		
			HIS-351	World Histor		
			MU-199	Music Video		
			PHY-101	Physical Prin		

department (학과)

course_id	prereq_id
BIO-301	BIO-101
BIO-399	BIO-101
CS-190	CS-101
CS-315	CS-101
CS-319	CS-101
CS-347	CS-101
EE-181	PHY-101

*prereq
(선수과목)*

section (개설강좌)

course_id	sec_id	semester	year	building	room_number	time_slot_id
BIO-101	1	Summer	2009	Painter	514	B
BIO-301	1	Summer	2010	Painter	514	A
CS-101	1	Fall	2009	Packard	101	H
CS-101	1	Spring	2010	Packard	101	F
CS-190	1	Spring	2009	Taylor	3128	E
CS-190	2	Spring	2009	Taylor	3128	A
CS-315	1	Spring	2010	Watson		
CS-319	1	Spring	2010	Watson		
CS-319	2	Spring	2010	Taylor		
CS-347	1	Fall	2009	Taylor		
EE-181	1	Spring	2009	Taylor		
FIN-201	1	Spring	2010	Packard		
HIS-351	1	Spring	2010	Painter		
MU-199	1	Spring	2010	Packard		
PHY-101	1	Fall	2009	Watson		

*teaches
(담당교수)*

Keys (키)

- 테이블 내 레코드를 특정하기 위한 방법
- 식별자(identifier) 역할을 할 수 있는 컬럼
 - ▣ 각 레코드를 유일하게 식별해낼 수 있는 성질
 - ▣ 개별 컬럼
 - 주민등록번호, 학번
 - ▣ 여러 개 컬럼의 조합
 - <주소, 이름>, <브랜드, 상품명, 용량>

instructor

ID	name	dept_name	salary
22222	Einstein	Physics	95000
12121	Wu	Finance	90000
32343	El Said	History	60000
45565	Katz	Comp. Sci.	75000
98345	Kim	Elec. Eng.	80000
76766	Crick	Biology	72000
10101	Srinivasan	Comp. Sci.	65000
58583	Califieri	History	62000
83821	Brandt	Comp. Sci.	92000
15151	Mozart	Music	40000
33456	Gold	Physics	87000
76543	Singh	Finance	80000

course

course_id	title	dept_name	credits
BIO-101	Intro. to Biology	Biology	4
BIO-301	Genetics	Biology	4
BIO-399	Computational Biology	Biology	3
CS-101	Intro. to Computer Science	Comp. Sci.	4
CS-190	Game Design	Comp. Sci.	4
CS-315	Robotics	Comp. Sci.	3
CS-319	Image Processing	Comp. Sci.	3
CS-347	Database System Concepts	Comp. Sci.	3
EE-181	Intro. to Digital Systems	Elec. Eng.	3
FIN-201	Investment Banking	Finance	3
HIS-351	World History	History	3
MU-199	Music Video Production	Music	3
PHY-101	Physical Principles	Physics	4

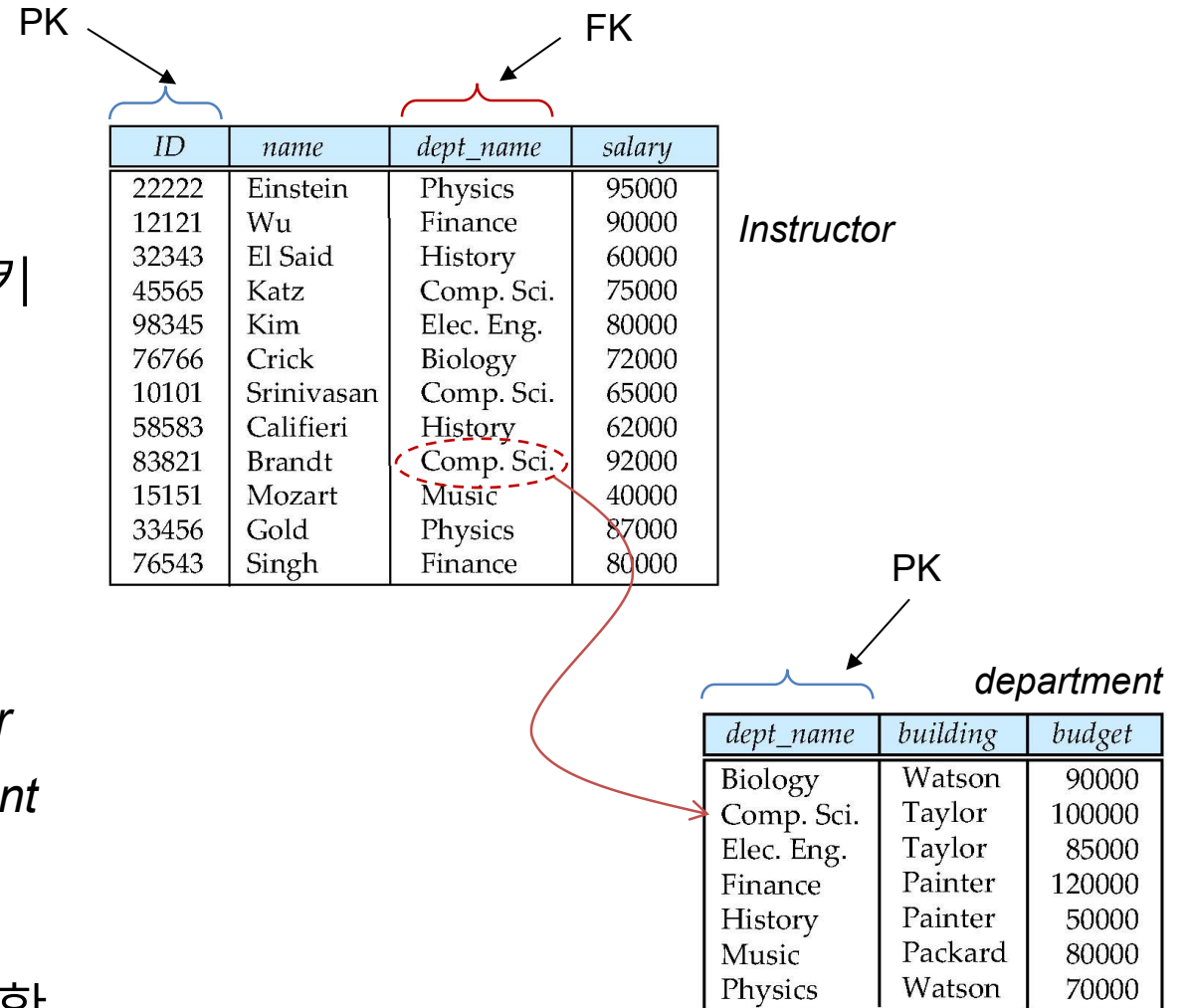
Keys

■ Primary key (기본키)

- 키 중에서 대표 식별자로 정의한 하나의 키
- 테이블마다 한 개

■ Foreign key (외래키)

- 다른 테이블의 기본키를 참조하는 컬럼
- **Referencing** (참조하는) relation: *instructor*
- **Referenced** (참조되는) relation: *department*
- *instructor.dept_name* 에는 *department.dept_name*에 나오는 값만 사용할 수 있음



instructor (교수)

ID	name	dept_name	salary
22222	Einstein	Physics	95000
12121	Wu	Finance	90000
32343	El Said	History	60000
45565	Katz	Comp. Sci.	75000
98345	Kim	Elec. Eng.	80000
76766	Crick	Biology	72000
10101	Srinivasan	Comp. Sci.	65000
58583	Califieri	History	62000
83821	Brandt	Comp. Sci.	92000
15151	Mozart	Music	40000
22456	Gold	Physics	87000
33456	Sh	Finance	80000

course (교과목)

course_id	title	dept_name	credits
BIO-101	Intro. to Biology	Biology	4
BIO-301	Genetics	Biology	4
BIO-399	Computational Biology	Biology	3
CS-101	Intro. to Computer Science	Comp. Sci.	4
CS-190	Game Design	Comp. Sci.	4
CS-315	Robotics	Comp. Sci.	4
CS-319	Image I	Comp. Sci.	4
CS-347	Databa	Comp. Sci.	4
EE-181	Intro. to	Elec. Eng.	4
FIN-201	Investm	Finance	4
HIS-351	World I	History	4
MU-199	Music V	Music	4
PHY-101	Physica	Physics	4

dept_name	building	budget
Biology	Watson	90000
Comp. Sci.	Taylor	100000
Elec. Eng.	Taylor	85000
Finance	Painter	120000
History	Painter	50000
Music	Packard	80000
Physics	Watson	70000

department (학과)

course_id	prereq_id
BIO-301	BIO-101
BIO-399	BIO-101
CS-190	CS-101
CS-315	CS-101
CS-319	CS-101
CS-347	CS-101
EE-181	PHY-101

prereq
(선수과목)

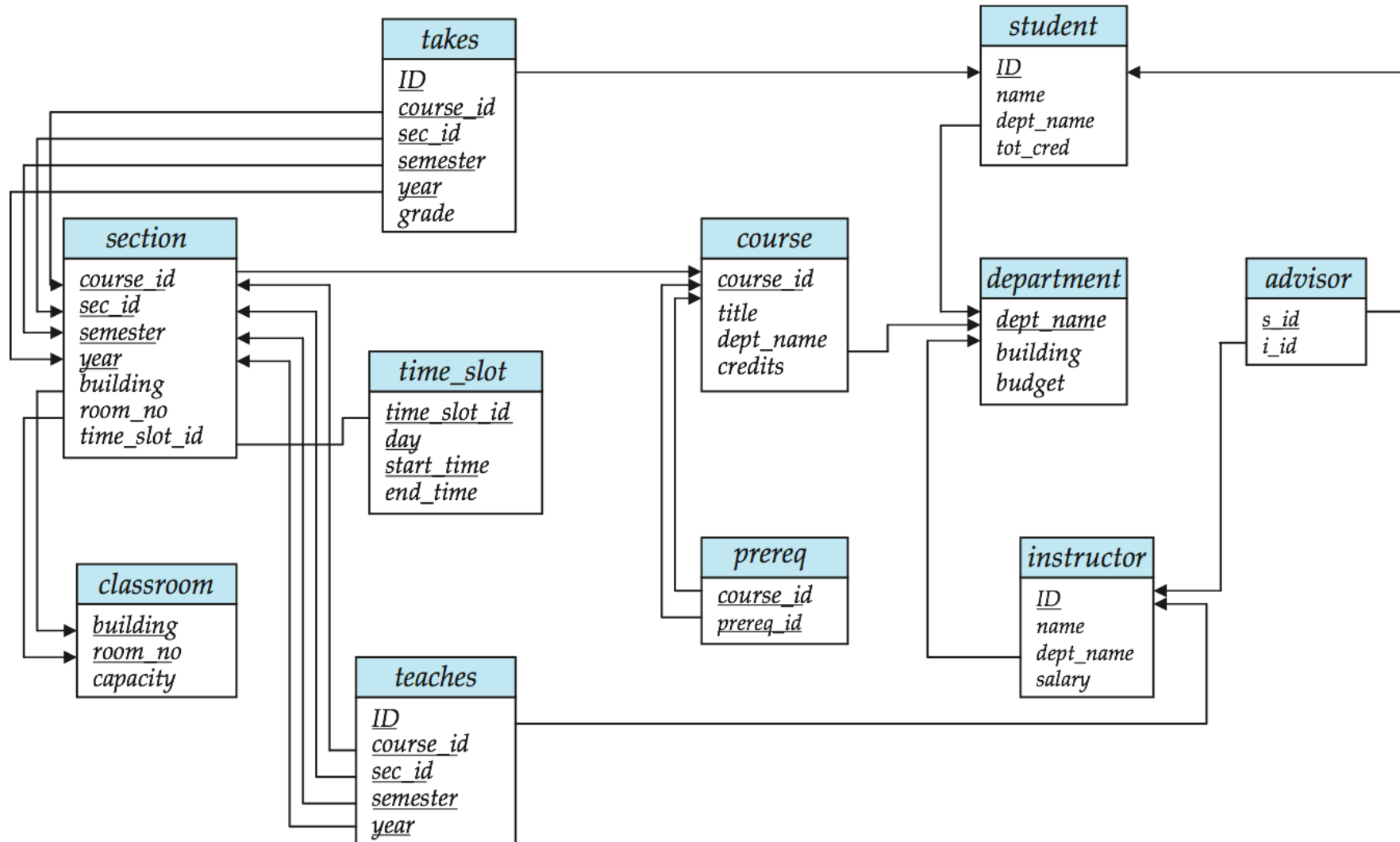
course_id	sec_id	semester	year	building	room_number	time_slot_id
BIO-101	1	Summer	2009	Painter	514	B
BIO-301	1	Summer	2010	Painter	514	A
CS-101	1	Fall	2009	Packard	101	H
CS-101	1	Spring	2010	Pac		
CS-190	1	Spring	2009	Tay		
CS-190	2	Spring	2009	Tay		
CS-315	1	Spring	2010	Wa		
CS-319	1	Spring	2010	Wa		
CS-319	2	Spring	2010	Tay		
CS-347	1	Fall	2009	Tay		
EE-181	1	Spring	2009	Tay		
FIN-201	1	Spring	2010	Pac		
HIS-351	1	Spring	2010	Pai		
MU-199	1	Spring	2010	Pac		
PHY-101	1	Fall	2009	Wa		

section (개설강좌)

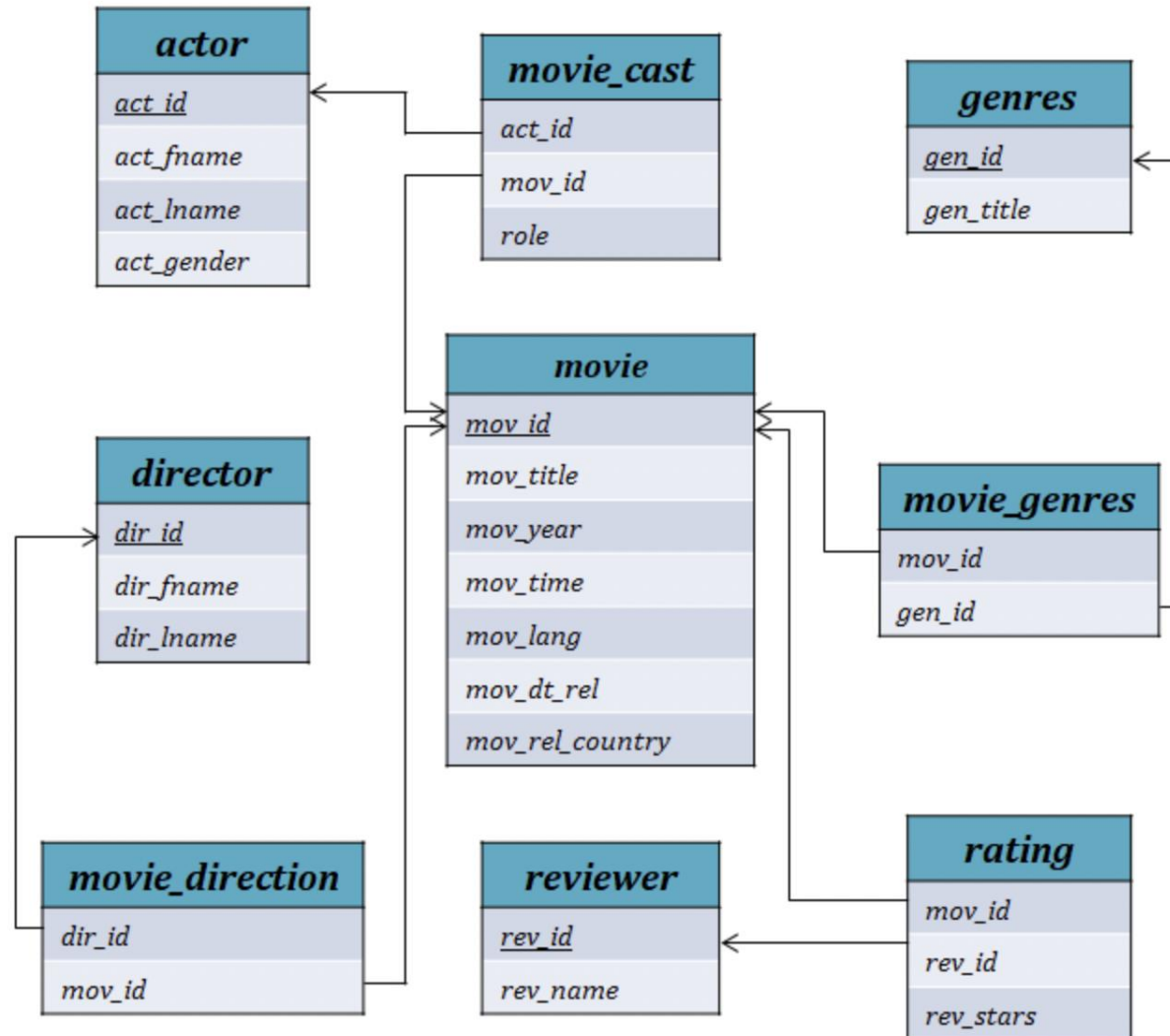
ID	course_id	sec_id	semester	year
10101	CS-101	1	Fall	2009
10101	CS-315	1	Spring	2010
10101	CS-347	1	Fall	2009
12121	FIN-201	1	Spring	2010
15151	MU-199	1	Spring	2010
22222	PHY-101	1	Fall	2009
32343	HIS-351	1	Spring	2010
45565	CS-101	1	Spring	2010
45565	CS-319	1	Spring	2010
76766	BIO-101	1	Summer	2009
76766	BIO-301	1	Summer	2010
83821	CS-190	1	Spring	2009
83821	CS-190	2	Spring	2009
83821	CS-319	2	Spring	2010
98345	EE-181	1	Spring	2009

teaches
(담당교수)

Schema Diagram for the University DB

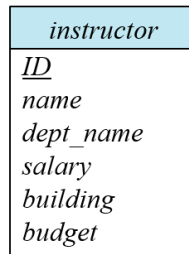


Schema Diagram for a Movie DB



Database Design

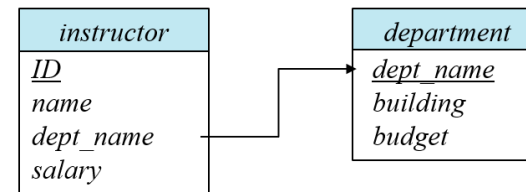
- 어떤 컬럼들을 가진 어떤 테이블들로 DB를 구성할 것인가?
 - 저장하려는 모든 정보를 표현(represent)할 수 있어야 하고
 - 중복을 최소로 하고
 - 운영/조작이 용이하도록



instructor

ID	name	dept_name	salary	building	budget
22222	Einstein	Physics	95000	Watson	70000
12121	Wu	Finance	90000	Painter	120000
32343	El Said	History	60000	Painter	50000
45565	Katz	Comp. Sci.	75000	Taylor	100000
98345	Kim	Elec. Eng.	80000	Taylor	85000
76766	Crick	Biology	72000	Watson	90000
10101	Srinivasan	Comp. Sci.	65000	Taylor	100000
58583	Califieri	History	62000	Painter	50000
83821	Brandt	Comp. Sci.	92000	Taylor	100000
15151	Mozart	Music	40000	Packard	80000
33456	Gold	Physics	87000	Watson	70000
76543	Singh	Finance	80000	Painter	120000

vs



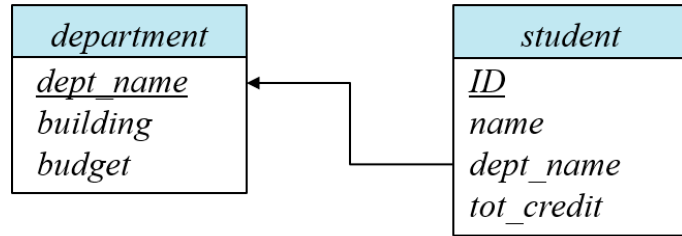
instructor

ID	name	dept_name	salary
10101	Srinivasan	Comp. Sci.	65000
12121	Wu	Finance	90000
15151	Mozart	Music	40000
22222	Einstein	Physics	95000
32343	El Said	History	60000
33456	Gold	Physics	87000
45565	Katz	Comp. Sci.	75000
58583	Califieri	History	62000
76543	Singh	Finance	80000
76766	Crick	Biology	72000
83821	Brandt	Comp. Sci.	92000
98345	Kim	Elec. Eng.	80000

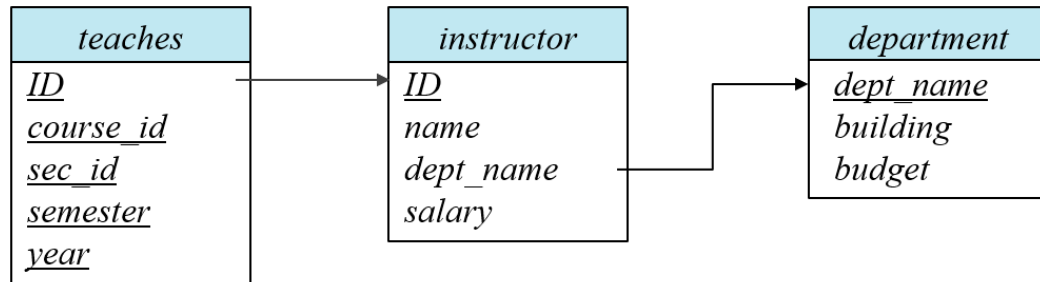
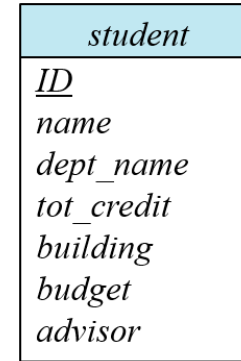
dept

dept_name	building	budget
Biology	Watson	90000
Comp. Sci.	Taylor	100000
Elec. Eng.	Taylor	85000
Finance	Painter	120000
History	Painter	50000
Music	Packard	80000
Physics	Watson	70000

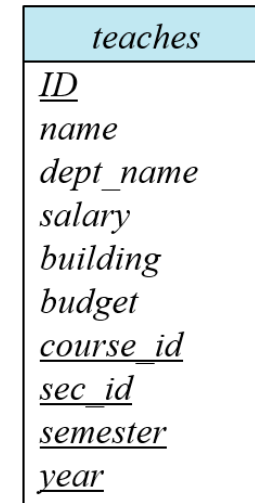
Database Design - 예제



VS



VS



3. SQL - 기본 (Basics)



질의 & 질의어

- 질의(query)
 - 질문
 - 데이터베이스에서 정보를 추출(extract/retrieve)하는 것
 - 정보요구 (information need)
- 질의어(query language)
 - 질의(정보요구)를 표현하는 언어
 - 일반적으로 광의로 사용 – 정보검색 뿐 아니라 삽입, 삭제, 수정 등 데이터 조작 연산(operation)에 대한 기능까지 포괄하는 데이터베이스 언어
- 절차적 언어 vs 비절차적 언어
- 이론적(순수) 언어 vs 시스템(구현) 언어

[예제] – 단일 테이블 질의

1) CS 학과 교수 다 찾아줘

```
select ID, name, dept_name, salary  
from instructor  
where dept_name = 'Comp. Sci.'
```

2) CS 학과 교수들 이름 보여줘

```
select name  
from instructor  
where dept_name = 'Comp. Sci.'
```

instructor

ID	name	dept_name	salary
22222	Einstein	Physics	95000
12121	Wu	Finance	90000
32343	El Said	History	60000
45565	Katz	Comp. Sci.	75000
98345	Kim	Elec. Eng.	80000
76766	Crick	Biology	72000
10101	Srinivasan	Comp. Sci.	65000
58583	Califieri	History	62000
83821	Brandt	Comp. Sci.	92000
15151	Mozart	Music	40000
33456	Gold	Physics	87000
76543	Singh	Finance	80000

[예제] – 다중 테이블 질의

- CS-101 과목을 강의한 교수들의 이름

```
select name
from instructor
where ID in ( select ID
               from teaches
               where course_id = 'CS-101' )
```

또는

```
select name
from instructor, teaches
where instructor.id = teaches.id and course_id = 'CS-101'
```

instructor

ID	name	dept_name	salary
10101	Srinivasan	Comp. Sci.	65000
12121	Wu	Finance	90000
15151	Mozart	Music	40000
22222	Einstein	Physics	95000
32343	El Said	History	60000

teaches

ID	course_id	sec_id	semester	year
10101	CS-101	1	Fall	2009
10101	CS-315	1	Spring	2010
10101	CS-347	1	Fall	2009
12121	FIN-201	1	Spring	2010
15151	MU-199	1	Spring	2010
22222	PHY-101	1	Fall	2009

SQL

- SQL (Structured Query Language, SQL 혹은 Sequel로 읽음)
 - 1970 E. F. Codd, relational model 제안
 - IBM이 **System R** 프로젝트로 relational DBMS 구현 (1974)
 - 그 질의어로 SEQUEL (**S**tructured **E**nglish **Q**Uery **L**anguage) 개발
- 표준 질의어로 채택되어 가장 널리 쓰이는 관계형 질의언어
 - 1986년 ANSI와 ISO에서 표준 질의어로 채택
 - 1992년 SQL2(SQL-92)
 - 2003년 SQL3 (객체지향적 요소 추가)

SQL

- DDL: Data Definition Language
 - 스키마의 생성, 수정, 삭제
 - **create table, drop table, alter table** 등의 명령문
- DML: Data Manipulation Language
 - 인스턴스(레코드 단위)에서의 삽입, 삭제, 변경 및 정보 질의
 - **insert**
 - **delete**
 - **update**
 - **select**
- **select** (질의) 문이 가장 많은 빈도로 사용됨

SELECT 문 (statement)

- 형식

select	< 컬럼 리스트 >
from	< 테이블 리스트 >
where	< 조건 >

- **select** 절 (clause)

- 질의 결과로 출력할 컬럼 리스트 지정

- **from** 절

- 질의 실행과정에 필요한 테이블들의 리스트

- **where** 절

- 검색되어야 하는 레코드에 대한 선택조건
- 생략 가능 – 모든 레코드

SELECT 절

- 교수들 이름과 소속학과

select *name, dept_name*
from *instructor*

<i>name</i>	<i>dept_name</i>
Srinivasan	Comp. Sci.
Wu	Finance
Mozart	Music
Einstein	Physics
El Said	History
Gold	Physics
Katz	Comp. Sci.
Califieri	History
Singh	Finance
Crick	Biology
Brandt	Comp. Sci.
Kim	Elec. Eng.

instructor

<i>ID</i>	<i>name</i>	<i>dept_name</i>	<i>salary</i>
10101	Srinivasan	Comp. Sci.	65000
12121	Wu	Finance	90000
15151	Mozart	Music	40000
22222	Einstein	Physics	95000
32343	El Said	History	60000
33456	Gold	Physics	87000
45565	Katz	Comp. Sci.	75000
58583	Califieri	History	62000
76543	Singh	Finance	80000
76766	Crick	Biology	72000
83821	Brandt	Comp. Sci.	92000
98345	Kim	Elec. Eng.	80000

- * (wild card)는 "all attributes"

select *
from *instructor*

<i>ID</i>	<i>name</i>	<i>dept_name</i>	<i>salary</i>
10101	Srinivasan	Comp. Sci.	65000
12121	Wu	Finance	90000
15151	Mozart	Music	40000
22222	Einstein	Physics	95000
32343	El Said	History	60000
33456	Gold	Physics	87000
45565	Katz	Comp. Sci.	75000
58583	Califieri	History	62000
76543	Singh	Finance	80000
76766	Crick	Biology	72000
83821	Brandt	Comp. Sci.	92000
98345	Kim	Elec. Eng.	80000

SELECT 절

- 교수들의 소속학과들은?

```
select dept_name  
from instructor
```

```
select distinct dept_name  
from instructor
```

dept_name
Comp. Sci.
Finance
Music
Physics
History
Physics
Comp. Sci.
History
Finance
Biology
Comp. Sci.
Elec. Eng.

dept_name
Comp. Sci.
Finance
Music
Physics
History
Biology
Elec. Eng.

instructor

ID	name	dept_name	salary
10101	Srinivasan	Comp. Sci.	65000
12121	Wu	Finance	90000
15151	Mozart	Music	40000
22222	Einstein	Physics	95000
32343	El Said	History	60000
33456	Gold	Physics	87000
45565	Katz	Comp. Sci.	75000
58583	Califieri	History	62000
76543	Singh	Finance	80000
76766	Crick	Biology	72000
83821	Brandt	Comp. Sci.	92000
98345	Kim	Elec. Eng.	80000

SELECT 절

- select절에 컬럼명 외에 산술식 사용이 가능
 - ▣ 컬럼명도 산술식의 한 종류임!
 - ▣ $+$, $-$, $*$, $/$
- 각 교수의 월급?

```
select ID, name, salary/12  
from instructor
```

ID	name	dept_name	salary
10101	Srinivasan	Comp. Sci.	65000
12121	Wu	Finance	90000
15151	Mozart	Music	40000
22222	Einstein	Physics	95000
32343	El Said	History	60000
33456	Gold	Physics	87000
45565	Katz	Comp. Sci.	75000
58583	Califieri	History	62000
76543	Singh	Finance	80000
76766	Crick	Biology	72000
83821	Brandt	Comp. Sci.	92000
98345	Kim	Elec. Eng.	80000

WHERE 절

- 각 레코드가 (결과에 나오기 위해) 만족해야 할 조건
 - **from** 절에 명시된 테이블의 레코드 하나 하나에 대해 평가

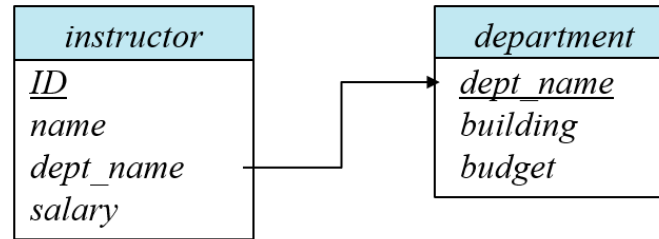
- 연봉 80000 이상인 CS 교수님 이름

```
select name
from instructor
where dept_name = 'Comp. Sci.'
       and salary >= 80000
```

- 비교 연산자: >, <, =, <=, >=, !=
- 논리 연산자: **and, or, not**

ID	name	dept_name	salary
10101	Srinivasan	Comp. Sci.	65000
12121	Wu	Finance	90000
15151	Mozart	Music	40000
22222	Einstein	Physics	95000
32343	El Said	History	60000
33456	Gold	Physics	87000
45565	Katz	Comp. Sci.	75000
58583	Califieri	History	62000
76543	Singh	Finance	80000
76766	Crick	Biology	72000
83821	Brandt	Comp. Sci.	92000
98345	Kim	Elec. Eng.	80000

[Exercise] 3 – 1



- 위 스키마에 대해, 아래 질의들에 해당하는 SQL문을 작성하시오.
 - A. 교수들의 이름들을 나열하시오.
 - B. 연봉이 90000을 넘는 교수들 이름.
 - C. "Taylor" 빌딩에 위치한 학과들은?

FROM 절

- 질의의 대상이 되는 테이블들 지정
- 테이블이 두 개일 경우
 - 양쪽 레코드간 만들 수 있는 모든 조합으로 대형 테이블 생성 – 카티션 곱 (Cartesian product)
 - where 절과 select 절은 이 대형 테이블에 대해 작용

<i>instructor</i>	<i>ID</i>	<i>name</i>	<i>dept_name</i>	<i>salary</i>						<i>teaches</i>					
	10101	Srinivasan	Comp. Sci.	65000											
	12121	Wu	Finance	90000											
	15151	Mozart	Music	40000											
	22222	Einstein	Physics	95000											
	32343	El Said	History												
				<i>inst.ID</i>	<i>name</i>	<i>dept_name</i>	<i>salary</i>	<i>teaches.ID</i>	<i>course_id</i>	<i>sec_id</i>	<i>semester</i>	<i>year</i>			
				10101	Srinivasan	Comp. Sci.	65000	10101	CS-101	1	Fall	2009	1	Fall	2009
				10101	Srinivasan	Comp. Sci.	65000	10101	CS-315	1	Spring	2010			
				10101	Srinivasan	Comp. Sci.	65000	10101	CS-347	1	Fall	2009			
				10101	Srinivasan	Comp. Sci.	65000	12121	FIN-201	1	Spring	2010			
				10101	Srinivasan	Comp. Sci.	65000	15151	MU-199	1	Spring	2010			
				10101	Srinivasan	Comp. Sci.	65000	22222	PHY-101	1	Fall	2009			
				...	...	...	...	...	...	...	...	...			
				...	...	...	...	...	...	...	...	...			
				12121	Wu	Finance	90000	10101	CS-101	1	Fall	2009			
				12121	Wu	Finance	90000	10101	CS-315	1	Spring	2010			
				12121	Wu	Finance	90000	10101	CS-347	1	Fall	2009			
				12121	Wu	Finance	90000	12121	FIN-201	1	Spring	2010			
				12121	Wu	Finance	90000	15151	MU-199	1	Spring	2010			
				12121	Wu	Finance	90000	22222	PHY-101	1	Fall	2009			
				...	...	...	...	...	...	...	...	...			
				...	...	...	...	...	...	...	...	...			

select *
from *instructor*, *teaches*

조인 (Join)

- 각 교수의 이름과 그가 가르친 과목 ID

```
select name, course_id
from instructor, teaches
where instructor.ID = teaches.ID
```

inst.ID	name	dept_name	salary	teaches.ID	course_id	sec_id	semester	year
10101	Srinivasan	Comp. Sci.	65000	10101	CS-101	1	Fall	2009
10101	Srinivasan	Comp. Sci.	65000	10101	CS-315	1	Spring	2010
10101	Srinivasan	Comp. Sci.	65000	10101	CS-347	1	Fall	2009
10101	Srinivasan	Comp. Sci.	65000	12121	FIN-201	1	Spring	2010
10101	Srinivasan	Comp. Sci.	65000	15151	MU-199	1	Spring	2010
10101	Srinivasan	Comp. Sci.	65000	22222	PHY-101	1	Fall	2009
...	...	...	...	...	...	...	...	...
...	...	...	...	...	...	...	...	...
12121	Wu	Finance	90000	10101	CS-101	1	Fall	2009
12121	Wu	Finance	90000	10101	CS-315	1	Spring	2010
12121	Wu	Finance	90000	10101	CS-347	1	Fall	2009
12121	Wu	Finance	90000	12121	FIN-201	1	Spring	2010
12121	Wu	Finance	90000	15151	MU-199	1	Spring	2010
12121	Wu	Finance	90000	22222	PHY-101	1	Fall	2009
...	...	...	...	...	...	...	...	...
...	...	...	...	...	...	...	...	...



- CS 교수들의 이름과 강의한 과목 ID

```
select name, course_id
from instructor, teaches
where instructor.ID = teaches.ID and
dept_name = 'Comp. Sci.'
```

inst.ID	name	dept_name	salary	teaches.ID	course_id	sec_id	semester	year
10101	Srinivasan	Comp. Sci.	65000	10101	CS-101	1	Fall	2009
10101	Srinivasan	Comp. Sci.	65000	10101	CS-315	1	Spring	2010
10101	Srinivasan	Comp. Sci.	65000	10101	CS-347	1	Fall	2009
12121	Wu	Finance	90000	12121	FIN-201	1	Spring	2010
15151	Mozart	Music	40000	15151	MU-199	1	Spring	2010
22222	Einstein	Physics	95000	22222	PHY-101	1	Fall	2009
32343	El Said	History	60000	32343	HIS-351	1	Spring	2010
45565	Katz	Comp. Sci.	75000	45565	CS-101	1	Spring	2010
45565	Katz	Comp. Sci.	75000	45565	CS-319	1	Spring	2010
76766	Crick	Biology	72000	76766	BIO-101	1	Summer	2009
76766	Crick	Biology	72000	76766	BIO-201	1	Summer	2010

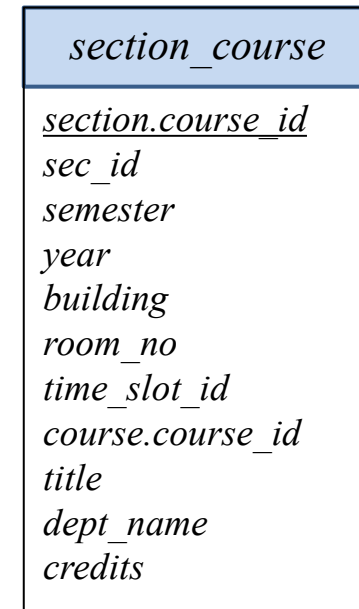
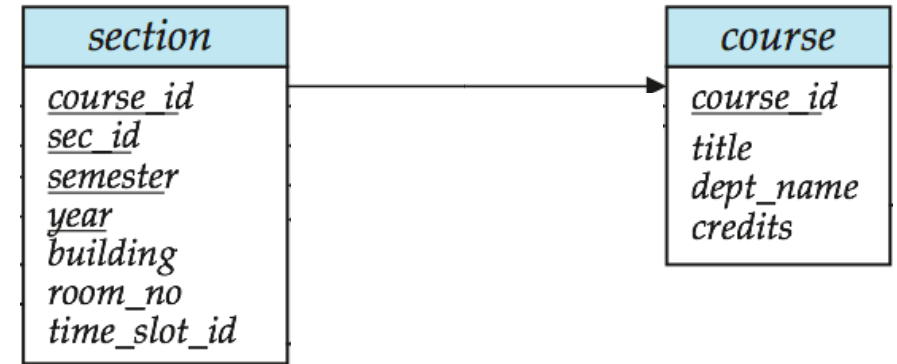
조인 (Join)

- 각 교과목명과 개설된 년도와 학기

```
select distinct title, year, semester  
from course, section  
where course.course_id = section.course_id
```

- 2021년에 개설된 CS 과목명

```
select distinct title  
from course, section  
where course.course_id = section.course_id  
       and year = '2021' and dept_name = 'Comp. Sci.'
```



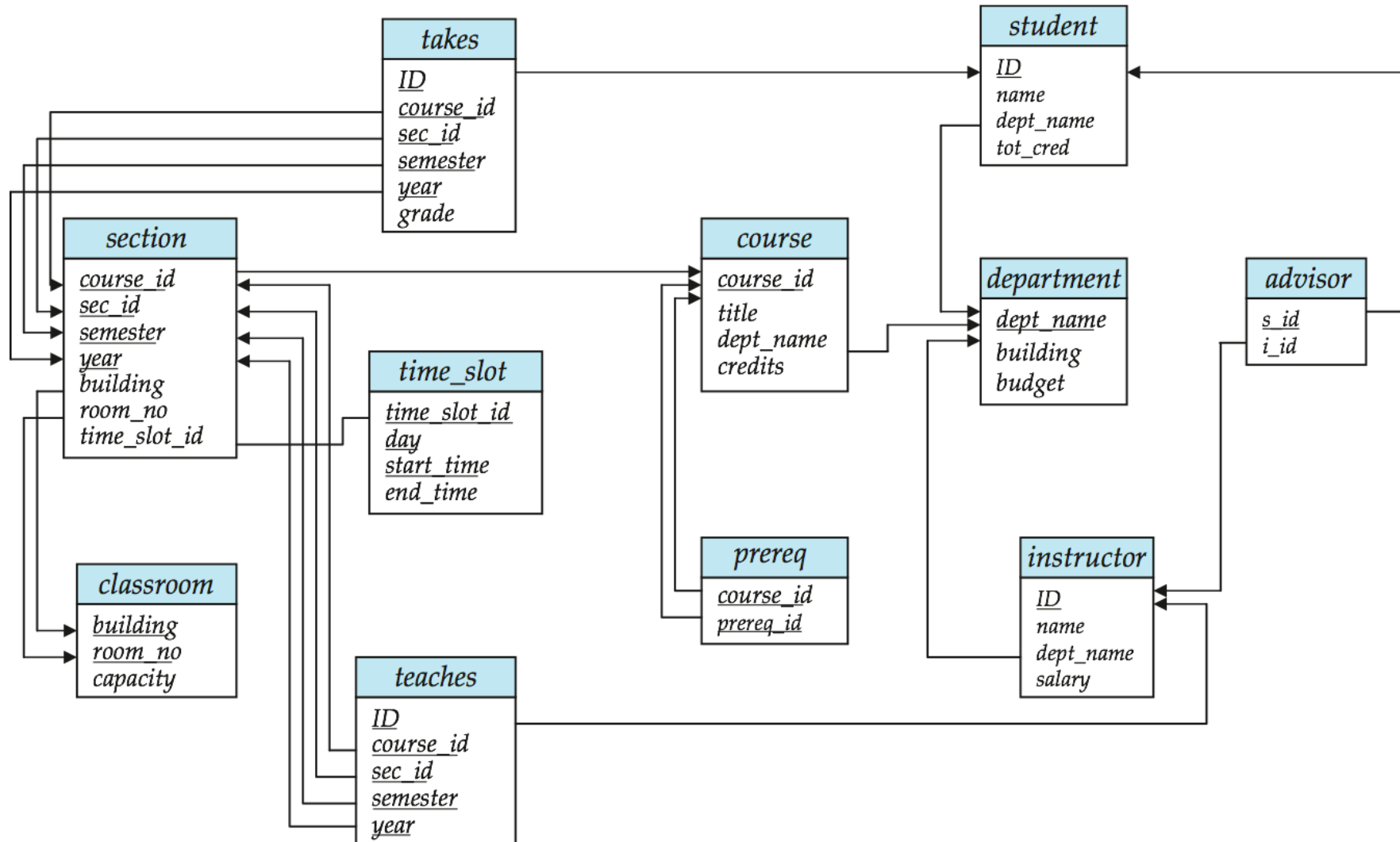
Rename

- From 절의 테이블들에 별명(alias)을 지정할 수 있음 - ***tuple variables***
 - 편의성
 - 같은 테이블을 조인할 때는 필수
- Find the names of all instructors who have a higher salary than professor 'Einstein'.

```
select distinct T.name
from instructor as T, instructor as S
where T.salary > S.salary and S.name = 'Einstein'
```

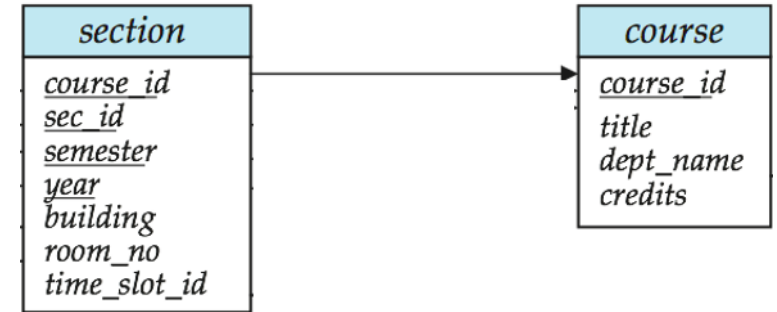
- **as** 는 생략 가능: *instructor* **as** *T* $\equiv$ *instructor* *T*

Schema Diagram for the University



[Exercise] 3 – 2

A. 각 교과목명과 개설된 년도와 학기



B. CS 학과 과목 중 2024년 2학기에 301동 건물에서 개설된 과목의 과목명과 학점

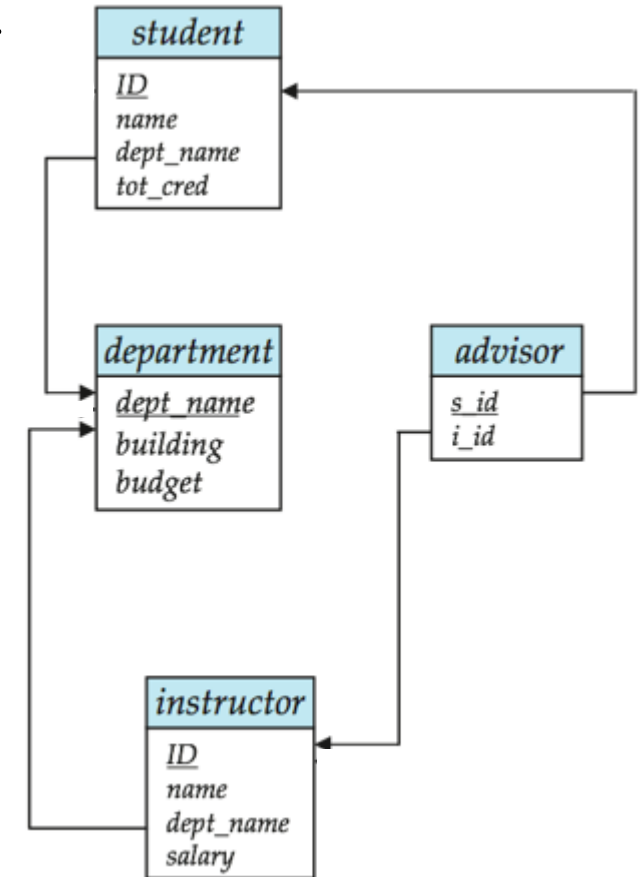
```
select S.name
from instructor I, advisor A, student S
where I.ID=A.i_id and A.s_id=S.ID
and I.name='Einstein'
```

[Exercise] 3 – 3

- 우측 스키마에 대해, 아래 질의들에 해당하는 SQL문을 작성하시오.

A. "CS" 학과와 같은 빌딩에 위치한 학과들의 학과명

B. "Einstein" 교수 지도학생들의 이름

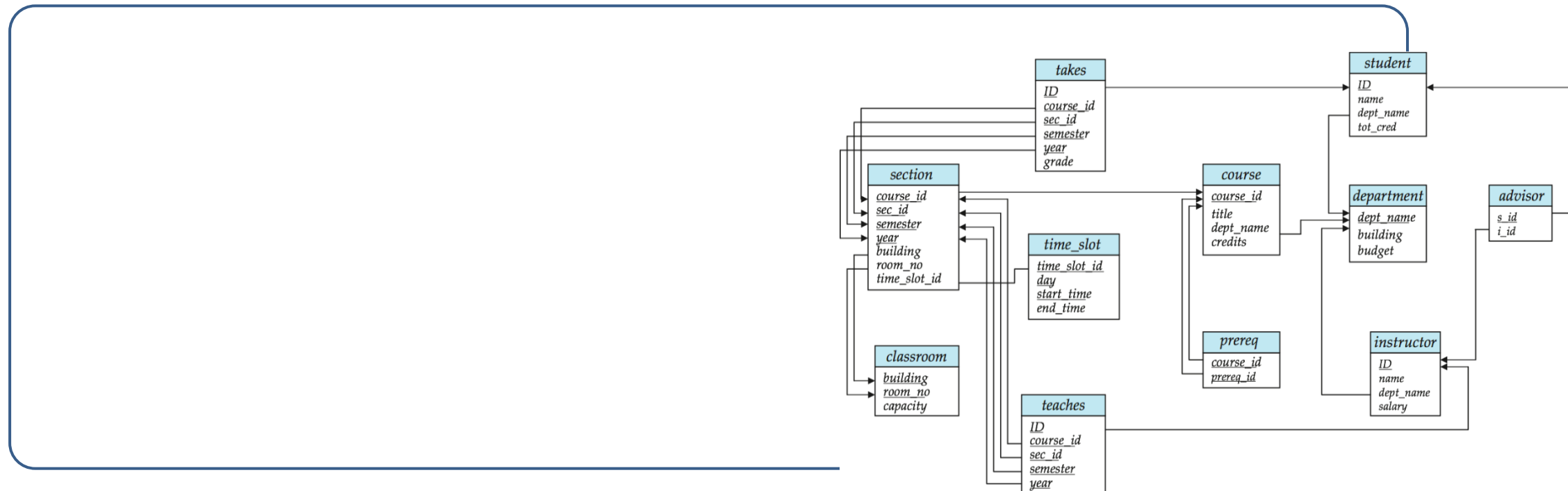


[Exercise] 3 – 4

- Einstein 교수님이 지도하는 학생들 ID

```
select s_id
from instructor, advisor
where ID=i_id and name='Einstein'
```

- A. Einstein 교수님이 가르쳤던(teach) 학생들 ID (결과에 중복이 없도록)



4. SQL – 집계 함수 (Aggregates)



[예제] – 간단한 집계

1) 교수들 평균 연봉

```
select avg( salary )  
from instructor
```

avg_salary
748333

2) CS 학과 교수 수

```
select count( ID )  
from instructor  
where dept_name = 'Comp. Sci.'
```

count id
3

instructor

ID	name	dept_name	salary
22222	Einstein	Physics	95000
12121	Wu	Finance	90000
32343	El Said	History	60000
45565	Katz	Comp. Sci.	75000
98345	Kim	Elec. Eng.	80000
76766	Crick	Biology	72000
10101	Srinivasan	Comp. Sci.	65000
58583	Califieri	History	62000
83821	Brandt	Comp. Sci.	92000
15151	Mozart	Music	40000
33456	Gold	Physics	87000
76543	Singh	Finance	80000

[예제] – 그룹 소계

1) 학과별 교수 평균 연봉

```
select dept_name, avg( salary )  
from instructor  
group by dept_name
```

dept_name	avg_salary
Biology	72000
Comp. Sci.	77333
Elec. Eng.	80000
Finance	85000
History	61000
Music	40000
Physics	91000

2) 학과별 교수 수

```
select dept_name, count( ID )  
from instructor  
group by dept_name
```

dept_name	count_id
Biology	1
Comp. Sci.	3
Elec. Eng.	1
Finance	2
History	2
Music	1
Physics	2

instructor

ID	name	dept_name	salary
22222	Einstein	Physics	95000
12121	Wu	Finance	90000
32343	El Said	History	60000
45565	Katz	Comp. Sci.	75000
98345	Kim	Elec. Eng.	80000
76766	Crick	Biology	72000
10101	Srinivasan	Comp. Sci.	65000
58583	Califieri	History	62000
83821	Brandt	Comp. Sci.	92000
15151	Mozart	Music	40000
33456	Gold	Physics	87000
76543	Singh	Finance	80000

집계함수 – Aggregate Functions

- 데이터 집합에서 하나의 대표값을 뽑아내는 함수
 - **avg**: average value
 - **min**: minimum value
 - **max**: maximum value
 - **sum**: sum of values
 - **count**: number of values
- **select** 절에 사용 – 산술식으로 취급
 - ▣ **select avg(salary), min(salary), max(salary)**
 - ▣ **select avg(grade) / 4.3 * 100**

집계함수 - 예제

1) 2021년 봄학기에 강의를 한 교수가 모두 몇 명이지?

```
select count ( distinct ID )
from teaches
where semester = 'Spring' and year = '2021'
```

2) 2021년 봄학기에 강의를 한 CS 교수가 모두 몇 명이지?

```
select count (distinct instructor.ID )
from teaches, instructor
where instructor.ID = teaches.ID and
semester = 'Spring' and year = '2021'
and dept_name = 'Comp. Sci.'
```

teaches

ID	course_id	sec_id	semester	year
10101	CS-101	1	Fall	2009
10101	CS-315	1	Spring	2010
10101	CS-347	1	Fall	2009
12121	FIN-201	1	Spring	2010
15151	MU-199	1	Spring	2010
22222	PHY-101	1	Fall	2009
32343	HIS-351	1	Spring	2010
45565	CS-101	1	Spring	2010
45565	CS-319	1	Spring	2010
76766	BIO-101	1	Summer	2009
76766	BIO-301	1	Summer	2010
83821	CS-190	1	Spring	2009
83821	CS-190	2	Spring	2009
83821	CS-319	2	Spring	2010
98345	EE-181	1	Spring	2009

instructor

ID	name	dept_name	salary
22222	Einstein	Physics	95000
12121	Wu	Finance	90000
32343	El Said	History	60000
45565	Katz	Comp. Sci.	75000
98345	Kim	Elec. Eng.	80000
76766	Crick	Biology	72000
10101	Srinivasan	Comp. Sci.	65000
58583	Califieri	History	62000
83821	Brandt	Comp. Sci.	92000
15151	Mozart	Music	40000
33456	Gold	Physics	87000
76543	Singh	Finance	80000

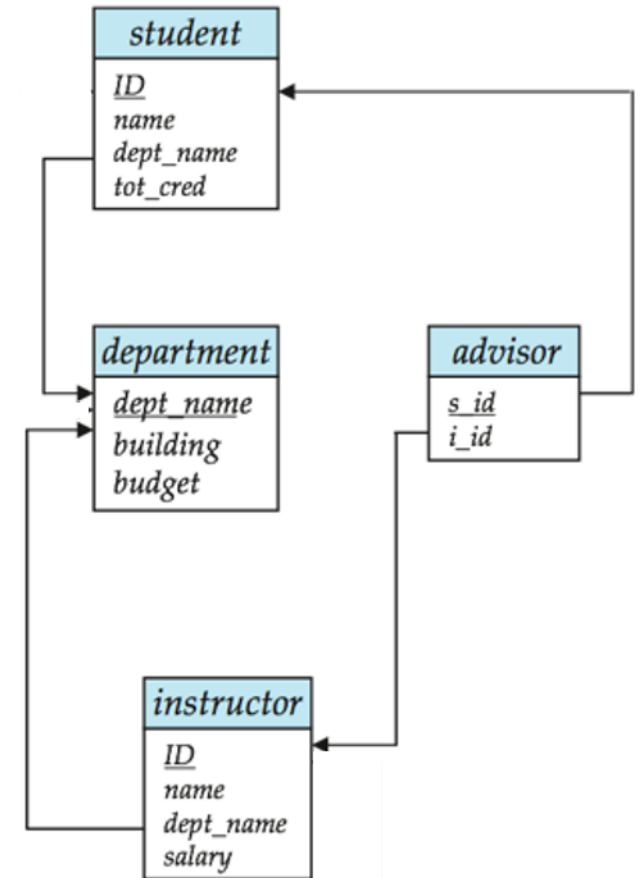
[Exercise] 4 – 1

A. 전체 학과 총 예산 및 (학과별) 평균 예산

B. Einstein 교수가 지도하는 학생들 수

(참고) Einstein 교수가 지도하는 학생들 ID

```
select s_id
from   instructor I, advisor A
where  I.ID = A.i_id and I.name = 'Einstein'
```



GROUP BY

- 지정된 컬럼 값이 같은 레코드들끼리 그룹을 지어 그룹별로 집계 함수 적용

- 위치는 where 절 다음

- 학교별 교수 연봉 평균

```
select dept_name, avg (salary)
from instructor
group by dept_name
```

- 중요:
- group by 절에 나열된 컬럼들과
select 절에 집계되지 않은 컬럼들이
같아야 함

instructor

ID	name	dept_name	salary
22222	Einstein	Physics	95000
12121	Wu	Finance	90000
32343	El Said	History	60000
45565	Katz	Comp. Sci.	75000
98345	Kim	Elec. Eng.	80000
76766	Crick	Biology	72000
10101	Srinivasan	Comp. Sci.	65000
58583	Califieri	History	62000
83821	Brandt	Comp. Sci.	92000
15151	Mozart	Music	40000
33456	Gold	Physics	87000
76543	Singh	Finance	80000



ID	name	dept_name	salary
76766	Crick	Biology	72000
45565	Katz	Comp. Sci.	75000
10101	Srinivasan	Comp. Sci.	65000
83821	Brandt	Comp. Sci.	92000
98345	Kim	Elec. Eng.	80000
12121	Wu	Finance	90000
76543	Singh	Finance	80000
32343	El Said	History	60000
58583	Califieri	History	62000
15151	Mozart	Music	40000
33456	Gold	Physics	87000
22222	Einstein	Physics	95000



dept_name	avg_salary
Biology	72000
Comp. Sci.	77333
Elec. Eng.	80000
Finance	85000
History	61000
Music	40000
Physics	91000

ORDER BY

- 검색 결과를 정렬하여 출력하는 기능
 - 위치는 select 문 마지막

- 교수들을 이름 순으로 정렬

```
select *  
from instructor  
order by name
```

- 내림차순은 desc 를 붙여 줌
- 여러 컬럼에 대해 정렬 가능
 - **order by** *name desc*
 - **order by** *dept-name desc, salary asc*

instructor

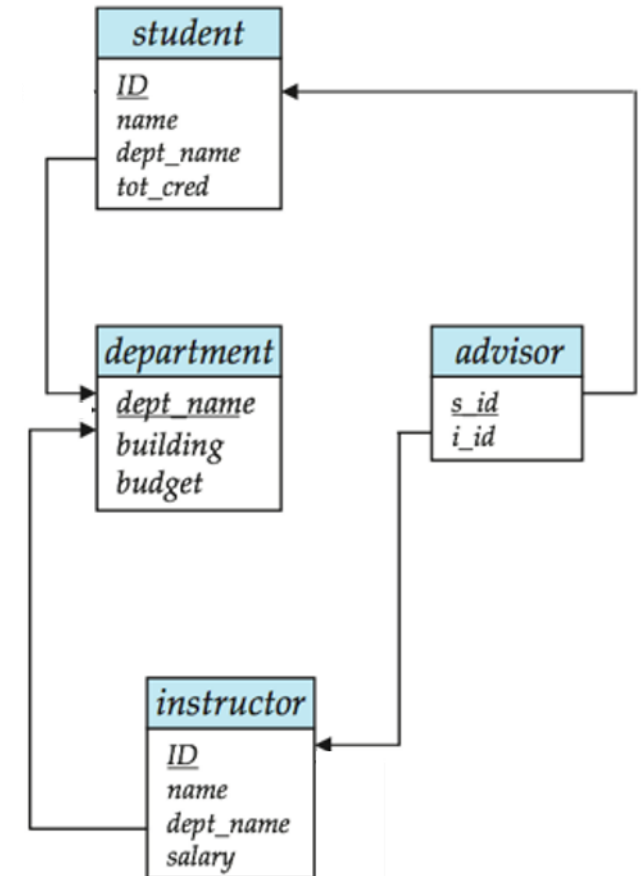
<i>ID</i>	<i>name</i>	<i>dept_name</i>	<i>salary</i>
10101	Srinivasan	Comp. Sci.	65000
12121	Wu	Finance	90000
15151	Mozart	Music	40000
22222	Einstein	Physics	95000
32343	El Said	History	60000
33456	Gold	Physics	87000
45565	Katz	Comp. Sci.	75000
58583	Califieri	History	62000
76543	Singh	Finance	80000
76766	Crick	Biology	72000
83821	Brandt	Comp. Sci.	92000
98345	Kim	Elec. Eng.	80000

[Exercise] 4 – 2

A. 학생들 이름을 오름차순으로

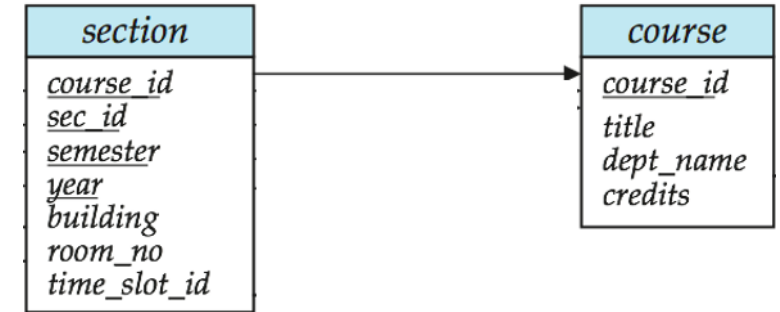
B. 교수들 정보를 학과명 오름차순, 연봉 내리차순으로

C. 교수(이름)별 지도학생 수



[Exercise] 4 – 3

A. 2021년 이후 연도별 총 강좌 수



B. 각 연도별 학과별 개설 강좌 수, 최근연도부터 학과명 순으로.

5. SQL – 조인 (Joins)



Join – On

- **ON** 절에 명시된 조건으로 양쪽 레코드 매칭하여 조인 수행

select *
from *instructor* **join** *teaches* **on** (*instructor.ID=teaches.ID*)

<i>instructor</i>				<i>teaches</i>				
<i>ID</i>	<i>name</i>	<i>dept_name</i>	<i>salary</i>	<i>ID</i>	<i>course_id</i>	<i>sec_id</i>	<i>semester</i>	<i>year</i>
10101	Srinivasan	Comp. Sci.	65000	10101	CS-101	1	Fall	2009
12121	Wu	Finance	90000	10101	CS-315	1	Spring	2010
15151	Mozart	Music	40000	10101	CS-347	1	Fall	2009
22222	Einstein	Physics	95000	12121	FIN-201	1	Spring	2010
32343	El Said	History	60000	15151	MU-199	1	Spring	2010
45565	Katz	Comp. Sci.	75000	22222	PHY-101	1	Fall	2009
76766	Crick	Biology	72000					
76766	Crick	Biology	72000					

<i>ID</i>	<i>name</i>	<i>dept_name</i>	<i>salary</i>	<i>ID</i>	<i>course_id</i>	<i>sec_id</i>	<i>semester</i>	<i>year</i>
10101	Srinivasan	Comp. Sci.	65000	10101	CS-101	1	Fall	2009
10101	Srinivasan	Comp. Sci.	65000	10101	CS-315	1	Spring	2010
10101	Srinivasan	Comp. Sci.	65000	10101	CS-347	1	Fall	2009
12121	Wu	Finance	90000	12121	FIN-201	1	Spring	2010
15151	Mozart	Music	40000	15151	MU-199	1	Spring	2010
22222	Einstein	Physics	95000	22222	PHY-101	1	Fall	2009
32343	El Said	History	60000	32343	HIS-351	1	Spring	2010
45565	Katz	Comp. Sci.	75000	45565	CS-101	1	Spring	2010
45565	Katz	Comp. Sci.	75000	45565	CS-319	1	Spring	2010
76766	Crick	Biology	72000	76766	BIO-101	1	Summer	2009
76766	Crick	Biology	72000	76766	BIO-301	1	Summer	2010

Join – On (cont.)

select * **from** r_1 **join** r_2 **on** *condition*;

≡

select * **from** r_1, r_2 **where** *condition*

- **from** 절에서 하나의 테이블로 취급

from $E_1, \dots, E_n$

E_i : table expression (테이블 식)

select distinct *name, title*

from *instructor* **join** *teaches* **on** (*instructor.ID=teaches.ID*), *course*

where *teaches.course_id=course.course_id*

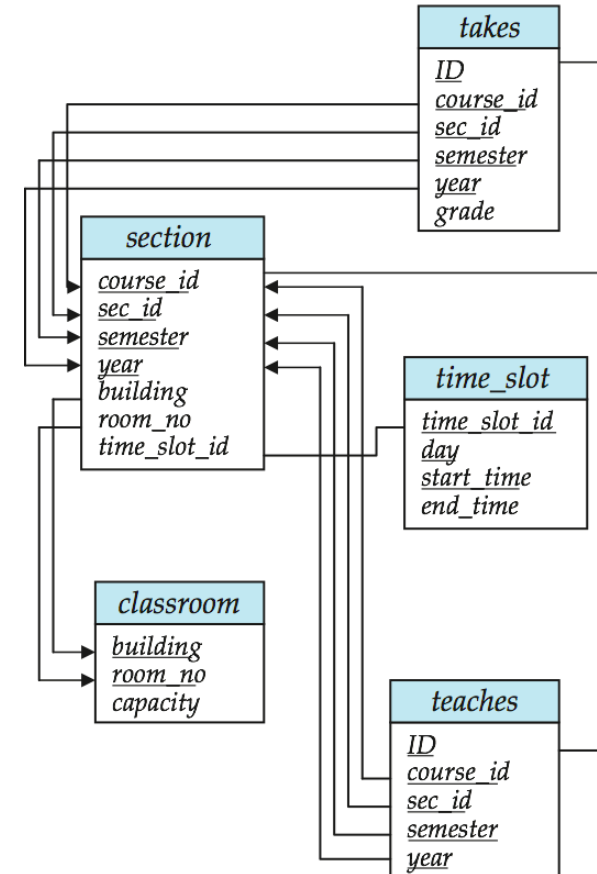
Join – Using

- 양쪽 테이블에 같은 이름의 컬럼으로 조인할 때

```
select *  
from section S, teaches T  
where S.course_id = T.course_id and  
       S.sec_id = T.sec_id and  
       S.semester = T.semester and  
       S.year = T.year
```

- Using 이용

```
select *  
from section join teaches using (course_id, sec_id, semester, year)
```



Join – Using

- ON
 - 조건을 명시적으로 제시 (**where** 절과 동일)
- USING
 - 동치(=; equality) 조건에 해당하는 컬럼 나열
 - 해당 컬럼은 양쪽 테이블 모두에 존재해야 함
 - 조인 후 해당 컬럼은 하나만 남음

```
select * from instructor inner join teaches using ( ID );
```

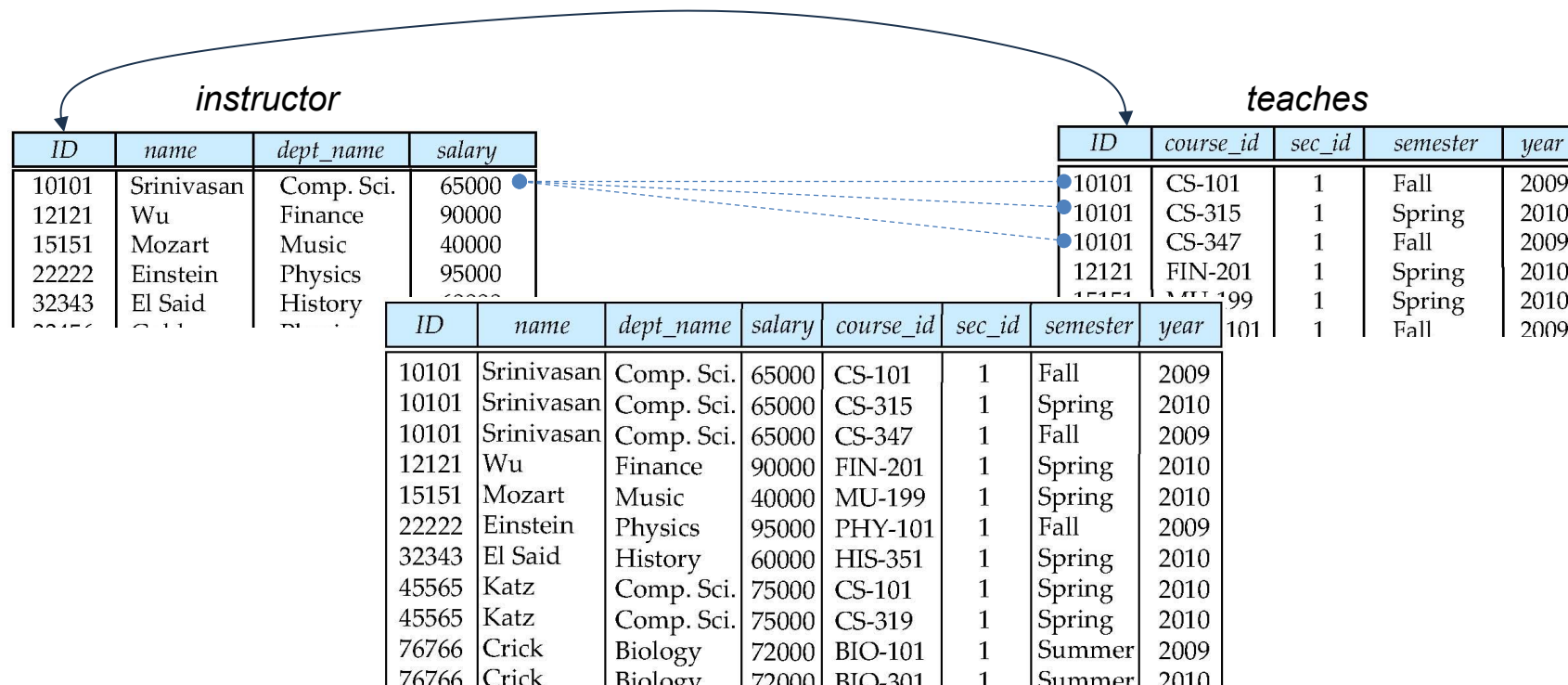
≡

```
select * from instructor join teaches on (instructor.ID=teaches.ID );
```

Natural Join

- 양쪽 테이블에 같은 이름의 컬럼들을 기준으로 조인

select *
from *instructor* natural join *teaches*



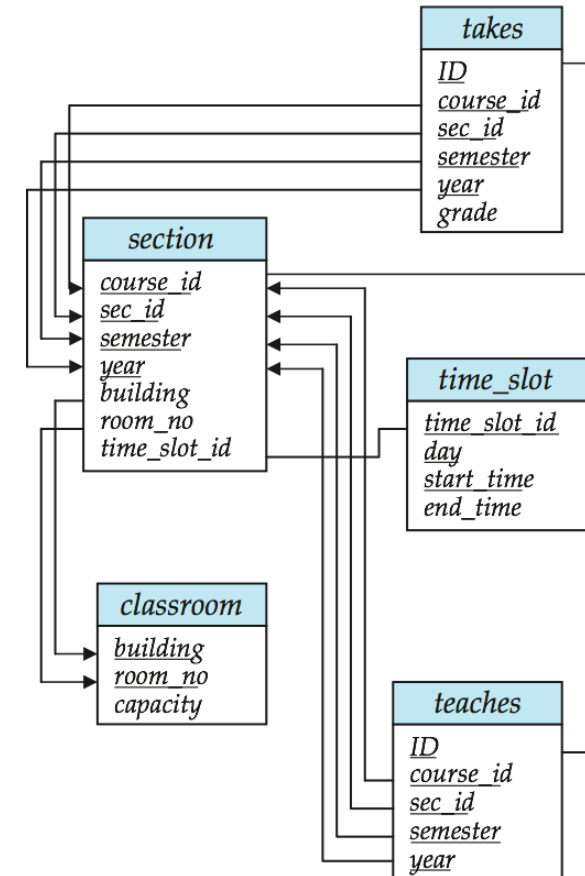
Natural Join

- 양쪽 테이블에 같은 이름의 컬럼으로 조인할 때.

```
select *  
from section S, teaches T  
where S.course_id = T.course_id and  
       S.sec_id = T.sec_id and  
       S.semester = T.semester and  
       S.year = T.year
```

- Natural Join 이용

```
select *  
from section natural join teaches
```

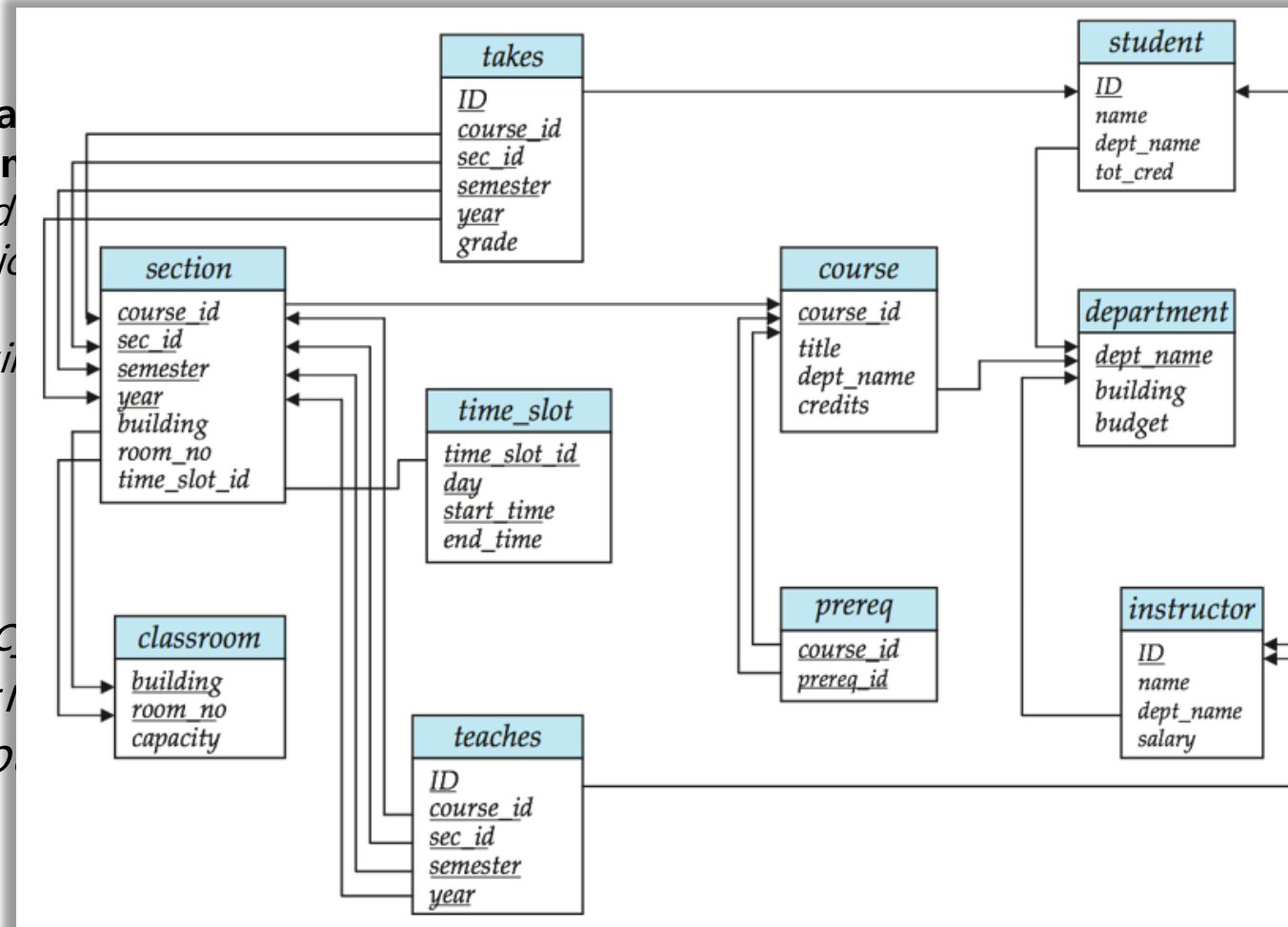


Joins

- Einstein 교수가 301동에서 강의한 과목을 들은 학생들 학번 (중복 제거)

select **distinct** *takes.ID*
from *takes, section, teaches, instructor*
where *takes.course_id=section.course_id and*
 takes.semester=section.semester and
 section.course_id=teaches.course_id and
 section.year=teaches.year and section
 teaches.ID=instructor.ID and
 instructor.name='Einstein' and bui

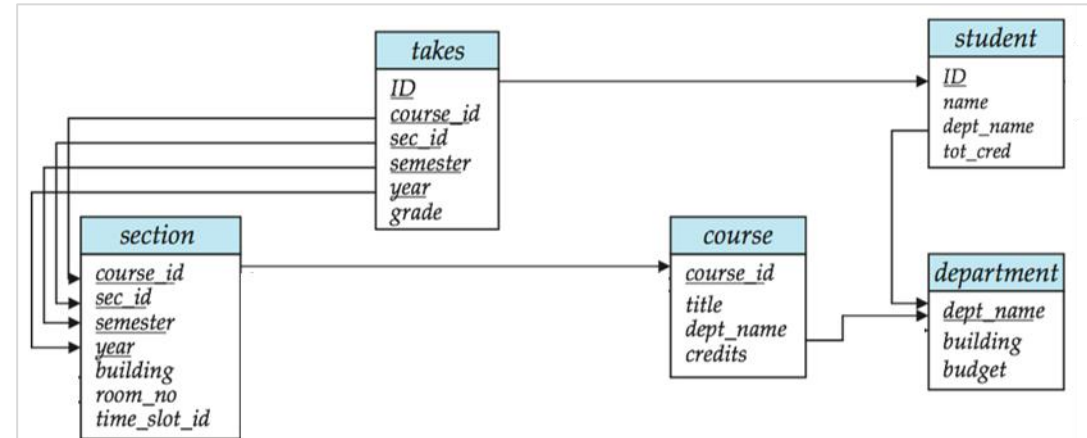
select **distinct** *takes.ID*
from (*takes natural join section*)
 join *teaches using* (*crse_id, sec*
 join *instructor on* (*teaches.ID=*
where *instructor.name='Einstein' and b*



[Exercise] 5 – 1

아래 질의에 해당하는 SQL문을 **join** 연산을 이용해 작성하시오.

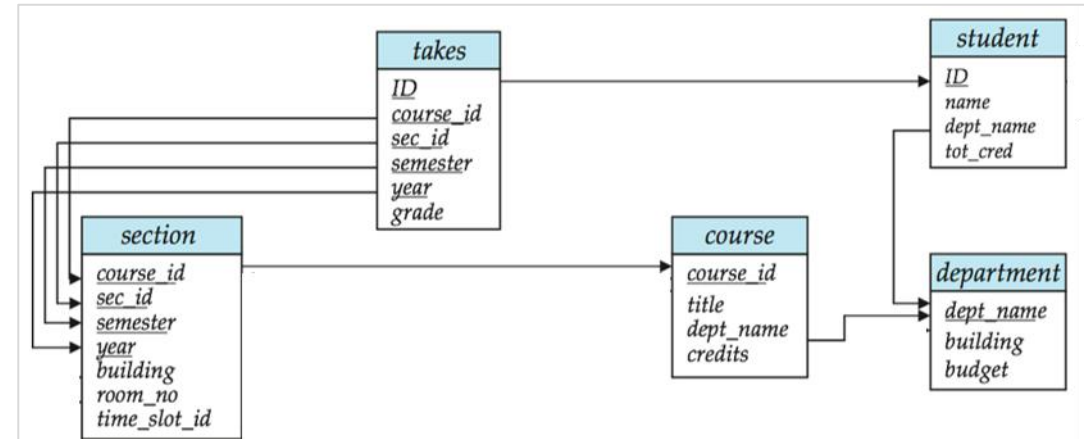
- 2020년에 CS 과목을 수강한 학생들 (ID & name)



[Exercise] 5 – 2

아래 질의에 해당하는 SQL문을 **join** 연산을 이용해 작성하시오.

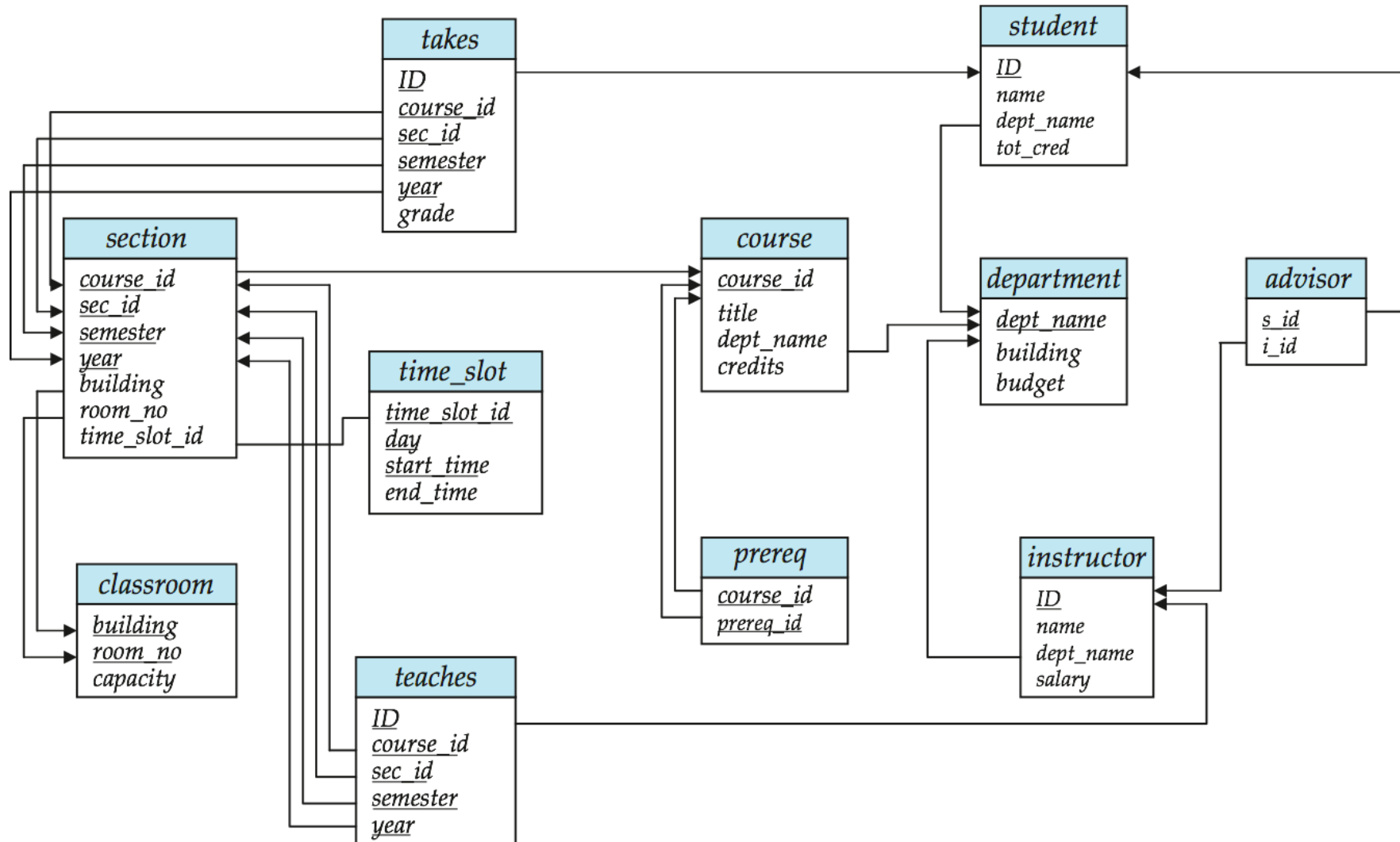
A. 년도별 CS 과목을 수강한 학생들 수



B. 년도별, 학과별 CS 과목을 수강한 학생들 수

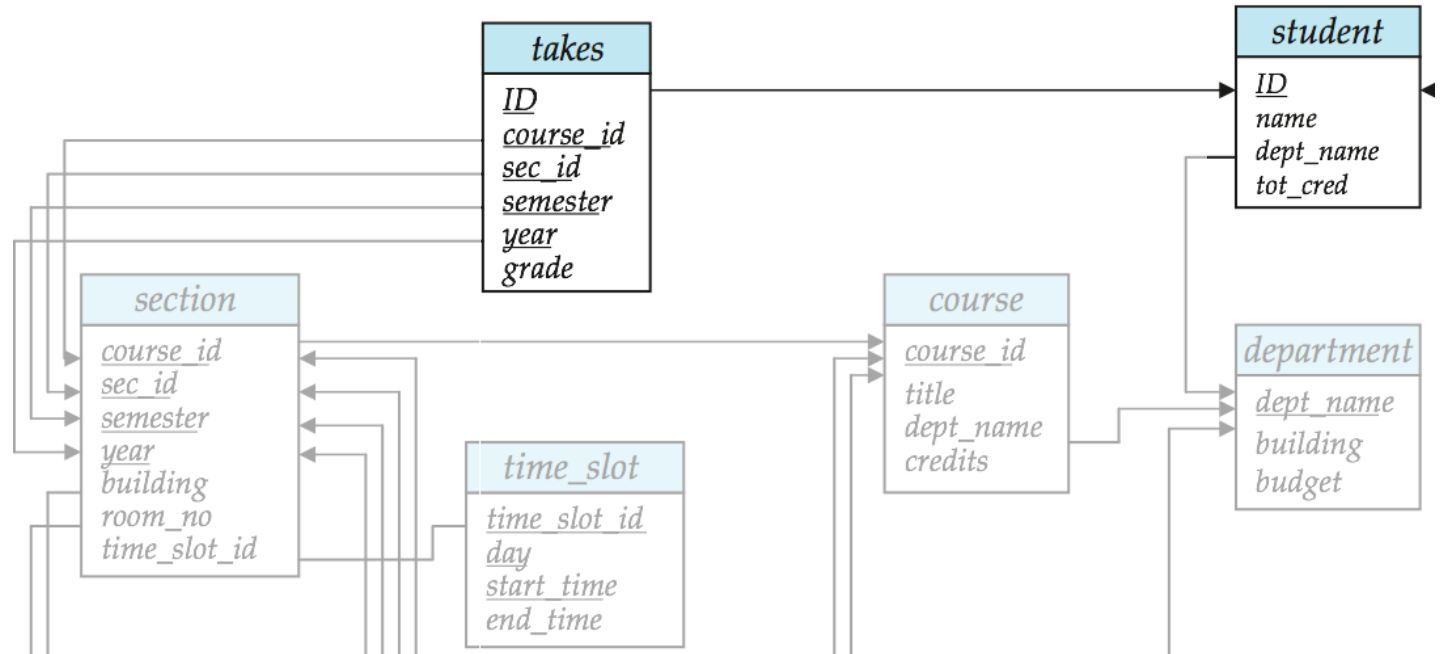
6. 데이터 분석

University DB



[예제] – (학생의) 학과별, 년도별 평점 평균

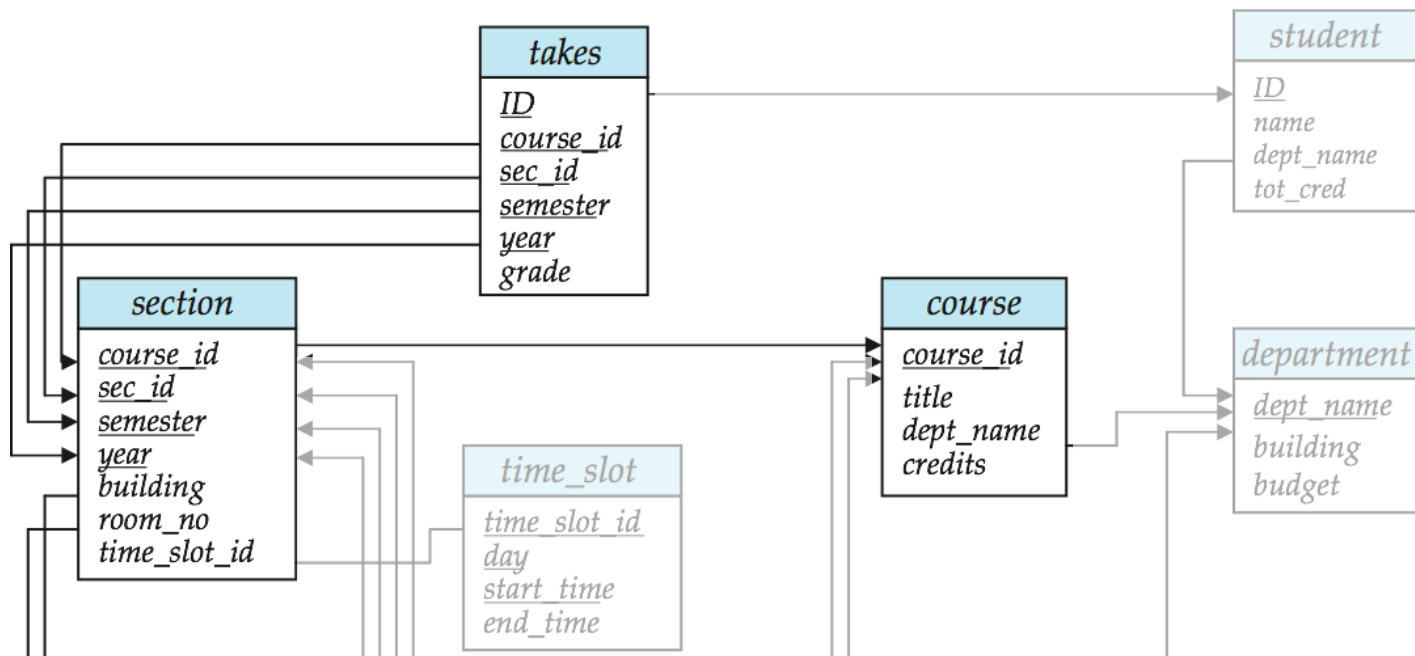
- <학과명, 년도, 평점평균>



```
select S.dept_name, T.year, avg (grade)
from takes T, student S
where T.ID = S.ID
group by S.dept_name, T.year
```

[예제] – (과목의) 학과별, 년도별 평점 평균

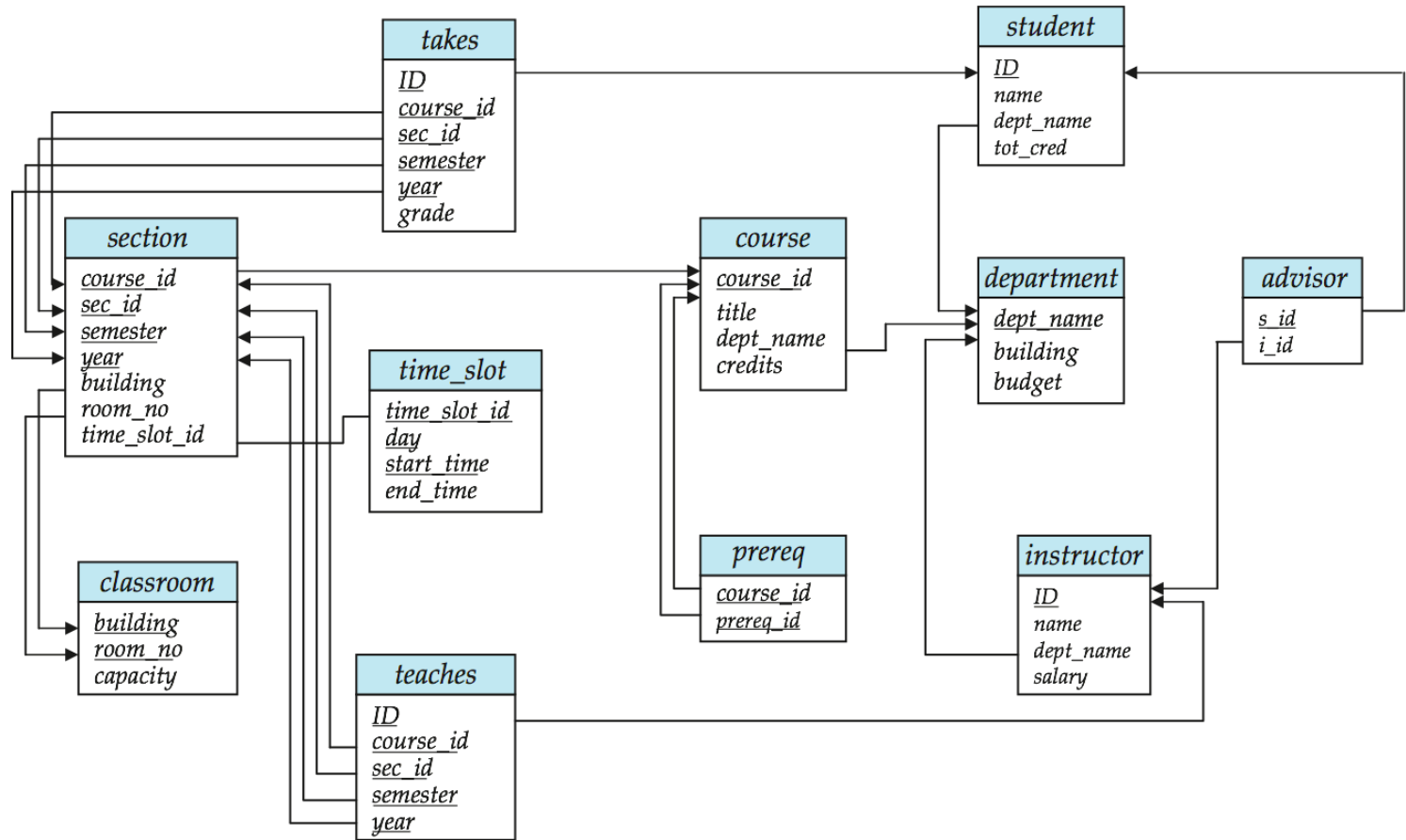
- <학과명, 년도, 평점평균>



```
select C.dept_name, T.year, avg (grade)
from takes T, course C
where T.course_id = C.course_id
group by C.dept_name, T.year
```

[예제] – 학사 분석

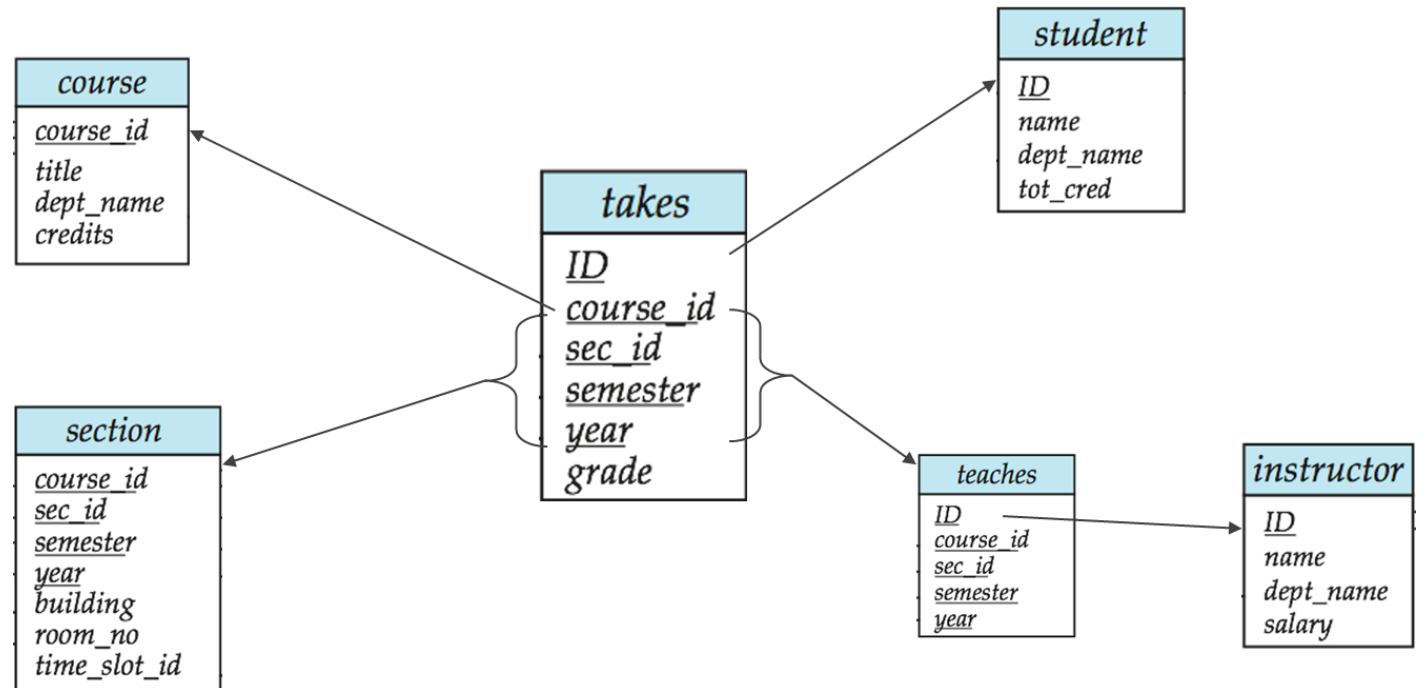
1. 학과별 교수별 년도별
평점평균
2. 교수 학과별 학생학과별
평점평균
3. 과목 학과별 학생 학과별
평점평균
4. 지도교수 과목 수강하는 평균
강좌 수



Star Schema

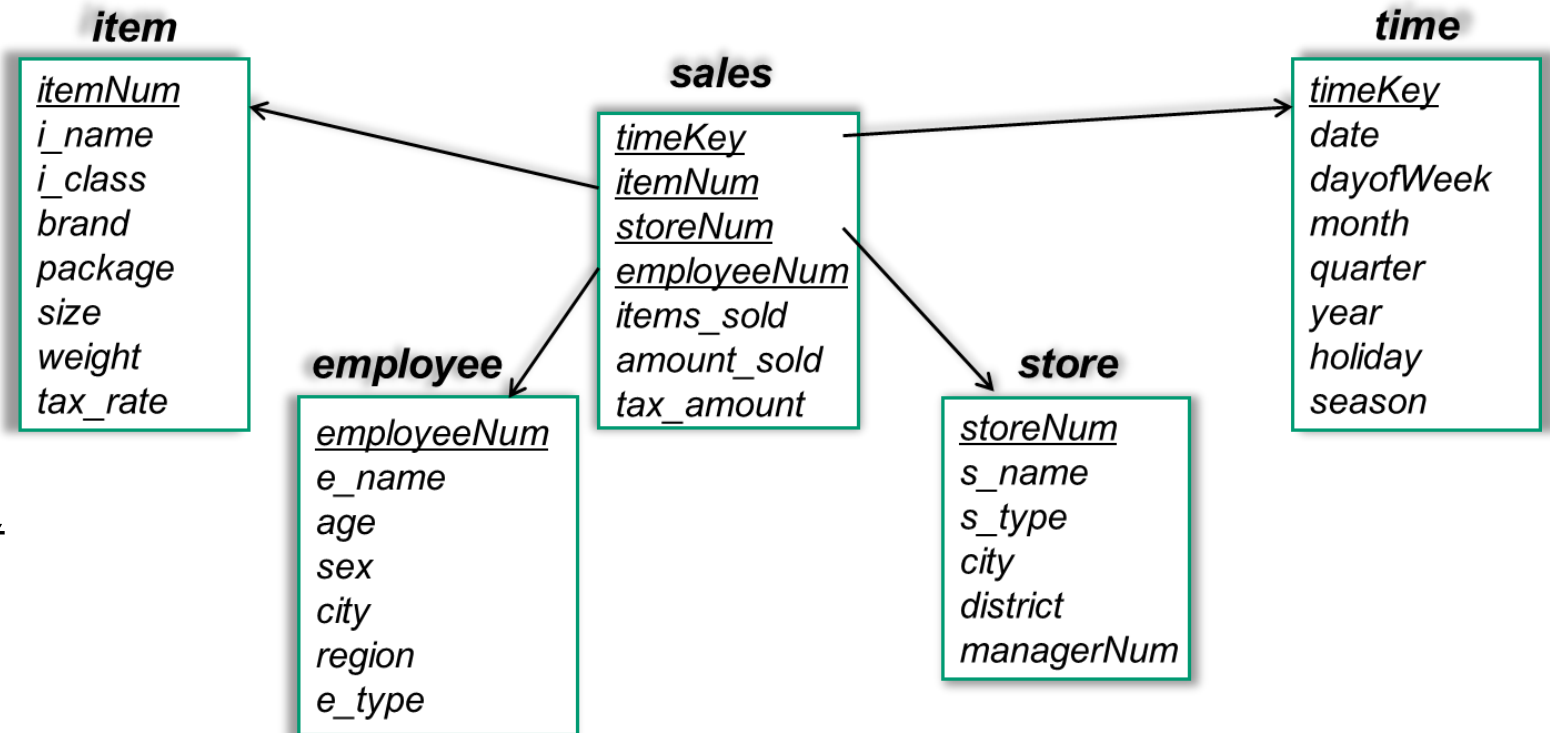
- DB 상의 어떤 event를 중심으로 참여 개체의 정보들을 FK로 연결

1. 학과별 교수별 년도별
평점평균
2. 교수 학과별 학생학과별
평점평균
3. 과목 학과별 학생 학과별
평점평균
4. 지도교수 과목 수강하는 평균
강좌 수



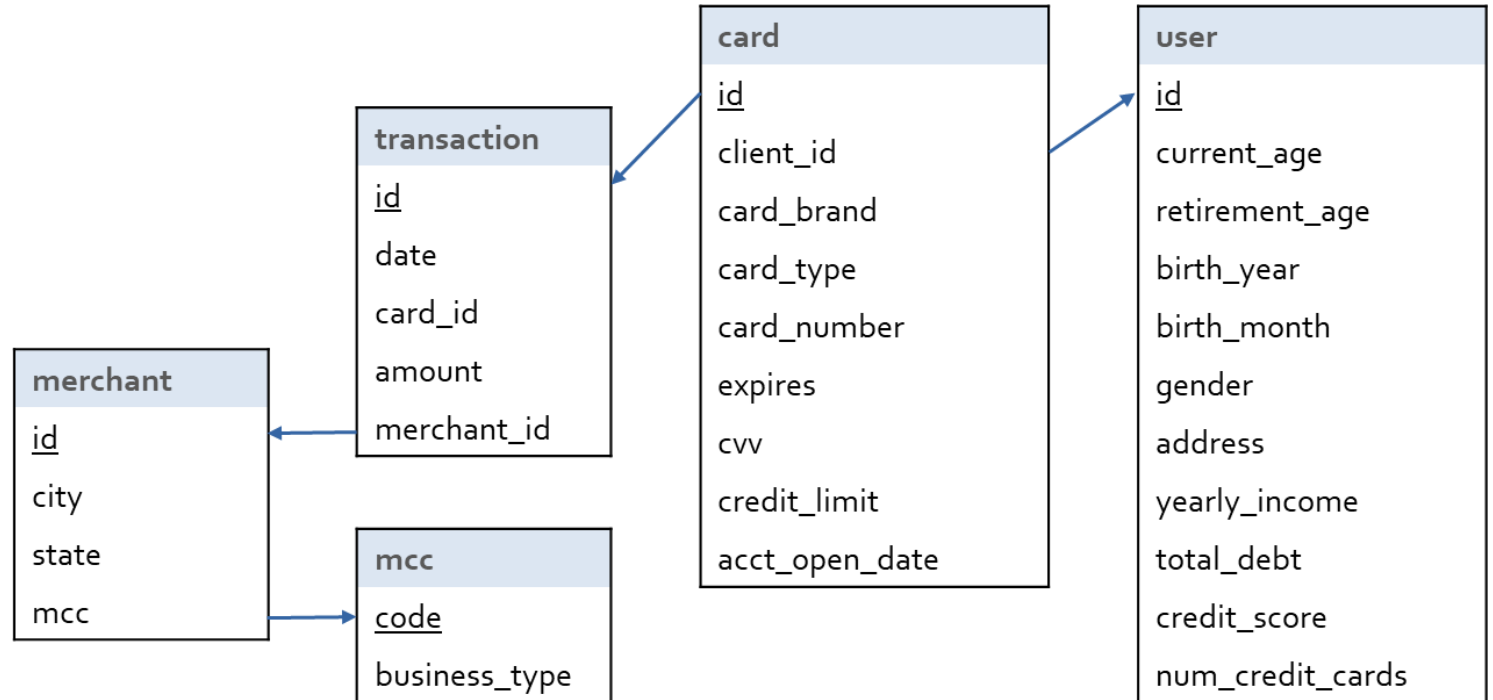
Star Schema – 편의점

1. 브랜드별 각 도시에서의 매출액
2. 지난 1분기 휴일에 휴게소 점포에서 가장 많이 팔린 음료
3. 년도별 점포종류별 여름 매출



Star Schema – 신용카드

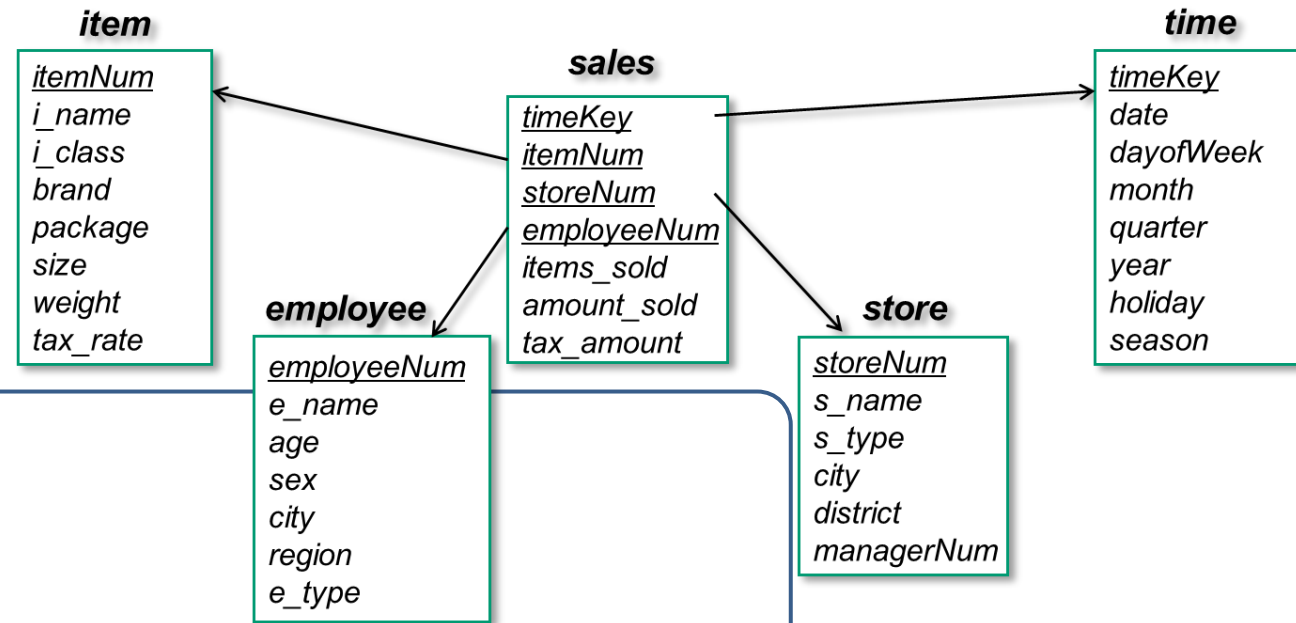
1. 30대 고객들이 주점에서 가장 많이 사용하는 카드 종류
2. 전체 거래 중 Visa 카드가 사용된 횟수와 비율
3. 신용점수 800 이상인 고객의 연소득 분포



[Exercise] 6 – 1

A. 각 brand 별로 각 도시에서의 총 매출액

B. 상품 종류(class) 별, 상점 유형(type) 별 겨울철 연매출



View – 가상 테이블

- 기존 테이블들로부터 생성되는 **가상의 테이블**

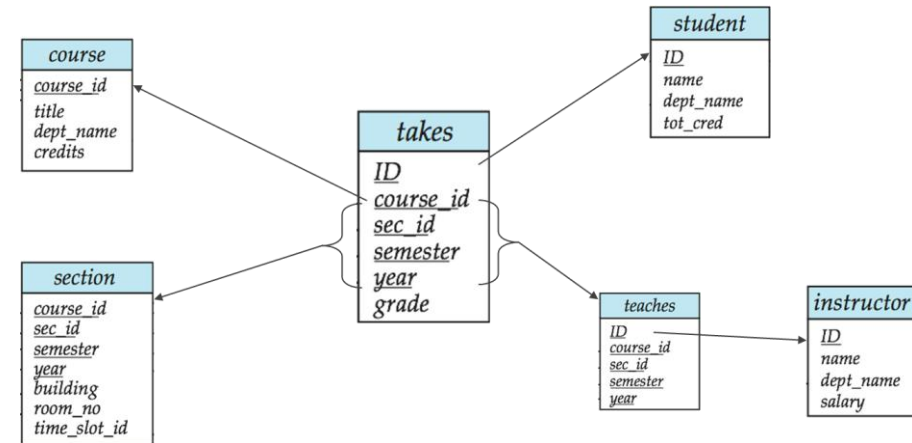
- 실제 데이터를 저장하는 물리적 테이블이 아니라 사용할 때마다 기존 테이블 데이터로 계산되어짐
- 사용자는 일반 테이블처럼 사용

```
create view instructor_2 as
  select ID, name, dept_name
  from instructor
```

- 기능

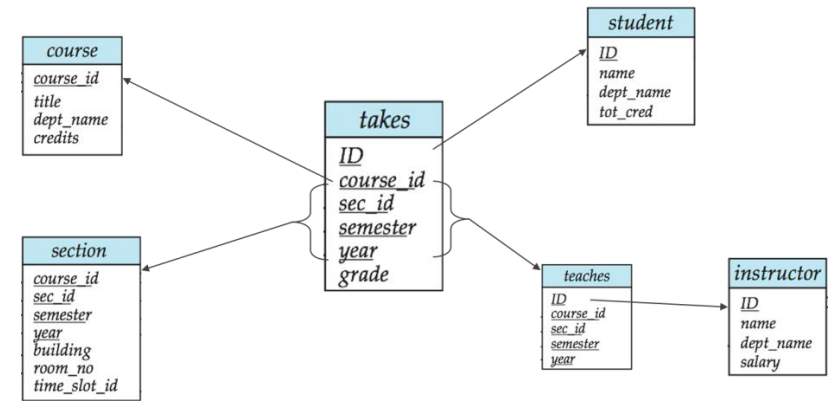
- 테이블의 내용 중 일부를 숨길 수 있음 - 보안의 효과
- 자주 사용하는 질의를 view로 만들어 관련 질의를 편리하게 표현

```
create view takes_detail as
  select *
  from (takes natural join section) natural join course
  join student using ( ID )
  join teaches using ( crse_id, sec_id, smster, year )
  join instructor on ( teaches.ID=instructor.ID )
```



View

- takes_detail view의 모습(가상)

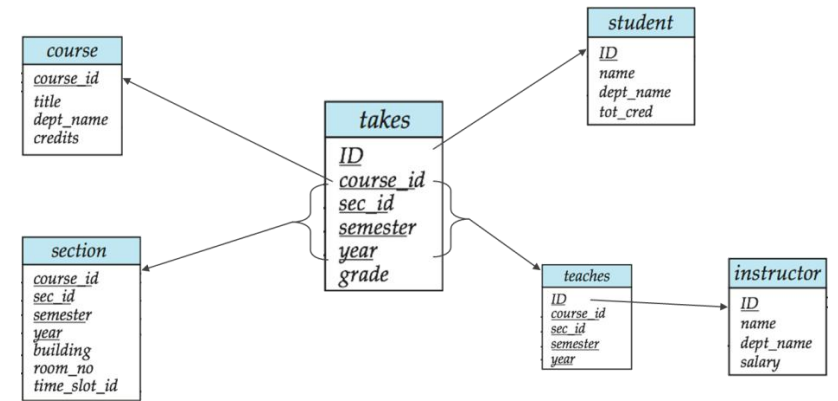


takes_detail

course			instructor				takes					student			
title	dept_name	credits	ID	name	dept_name	salary	course_id	sec_id	semester	year	grade	ID	name	dept_name	tot_cred
Intro. to Computer Science	Comp. Sci.	4	10101	Srinivasan	Comp. Sci.	65000	CS-101	1	Fall	2009	4.0	151	Wu	Finance	90000
Game Design	Comp. Sci.	4	10101	Srinivasan	Comp. Sci.	65000	CS-315	1	Spring	2010	3.7	2222	Mozart	Music	40000
Robotics	Comp. Sci.	3	10101	Srinivasan	Comp. Sci.	65000	CS-347	1	Fall	2009	3.4	2343	Einstein	Physics	95000
Investment Banking	Finance	3	12121	Wu	Finance	90000	FIN-201	1	Spring	2010	3.0	456	El Said	History	60000
World History	History	3	15151	Mozart	Music	40000	MU-199	1	Spring	2010	3.7	565	Gold	Physics	87000
Music Video Production	Music	3	22222	Einstein	Physics	95000	PHY-101	1	Fall	2009	3.0	65	Katz	Comp. Sci.	75000
Physical Principles	Physics	4	32343	El Said	History	60000	HIS-351	1	Spring	2010	2.8	73	Califieri	History	62500
Intro. to Computer Science	Comp. Sci.	4	45565	Katz	Comp. Sci.	75000	CS-101	1	Spring	2010	2.0				
Game Design	Comp. Sci.	4	45565	Katz	Comp. Sci.	75000	CS-319	1	Spring	2010	3.7				

View

■ takes_detail 완성



```

create view takes_detail as
select student.ID as sID, student.name as sname, student.dept_name as sdept,
       tot_cred, course_id, sec_id, semester, year, grade, title,
       course.dept_name as cdept, credits, building, room_no, time_slot_id,
       instructor.ID as iID, instructor.name as iname,
       instructor.dept_name as idept, salary
from (takes natural join section natural join course)
join student using ( ID )
join teaches using ( course_id, sec_id, semester, year )
join instructor on ( teaches.ID = instructor.ID )
    
```

1. 교수 학과별 학생학과별 평점평균

```

select idept, sdept, avg(grade)
from takes_detail
group by idept, sdept
    
```

2. CS 교수별 학생 학과별 평점평균

```

select iID, iname, sdept, avg(grade)
from takes_detail
where idept = 'Comp. Sci.'
group by iID, iname, sdept
    
```

takes_detail	
student	sID sname sdept tot_cred
takes	course_id sec_id semester year grade title
course	cdept credits building room_no time_slot_id
section	iID
instructor	iname idept salary

[Exercise] 6 – 2

- 우측 테이블에 대해 다음 질의에 해당하는 SQL문을 작성하시오.

A. CS 학생들이 어느 건물에 자주 갈까

B. CS 과목 수강 경험 학생이 가장 많은 타과

<i>takes_detail</i>	
student	<i>sID</i>
	<i>sname</i>
	<i>sdept</i>
	<i>tot_cred</i>
takes	<i>course_id</i>
	<i>sec_id</i>
	<i>semester</i>
	<i>year</i>
course	<i>grade</i>
	<i>title</i>
	<i>cdept</i>
	<i>credits</i>
section	<i>building</i>
	<i>room_no</i>
	<i>time_slot_id</i>
	<i>iID</i>
instructor	<i>iname</i>
	<i>idept</i>
	<i>salary</i>

7. DB for ML



DB for ML

- 기계학습 (ML; Machine Learning)
 - 과거(기존) 데이터를 학습하여 모델을 만들고
 - 그 모델을 이용해 새로운 입력에 대해 결과를 예측하거나 생성
- Feature engineering
 - 원시 데이터(raw data)로부터 전처리와 변환 과정을 거쳐 **ML이 사용하는 형태의 데이터** 구축
 - features

customer – feature table

customer_id	total_purch_7d	total_purch_30d
1	25	350
2	8	120
3	40	200

product – feature table

product_id	category
A	clothes
B	shoes
C	sports

Input
label -
feature
table

customer_id	product_id	on_sale	purchased
1	A	Y	Y
1	B	Y	N
2	C	N	Y
3	A	N	Y

Join + etc

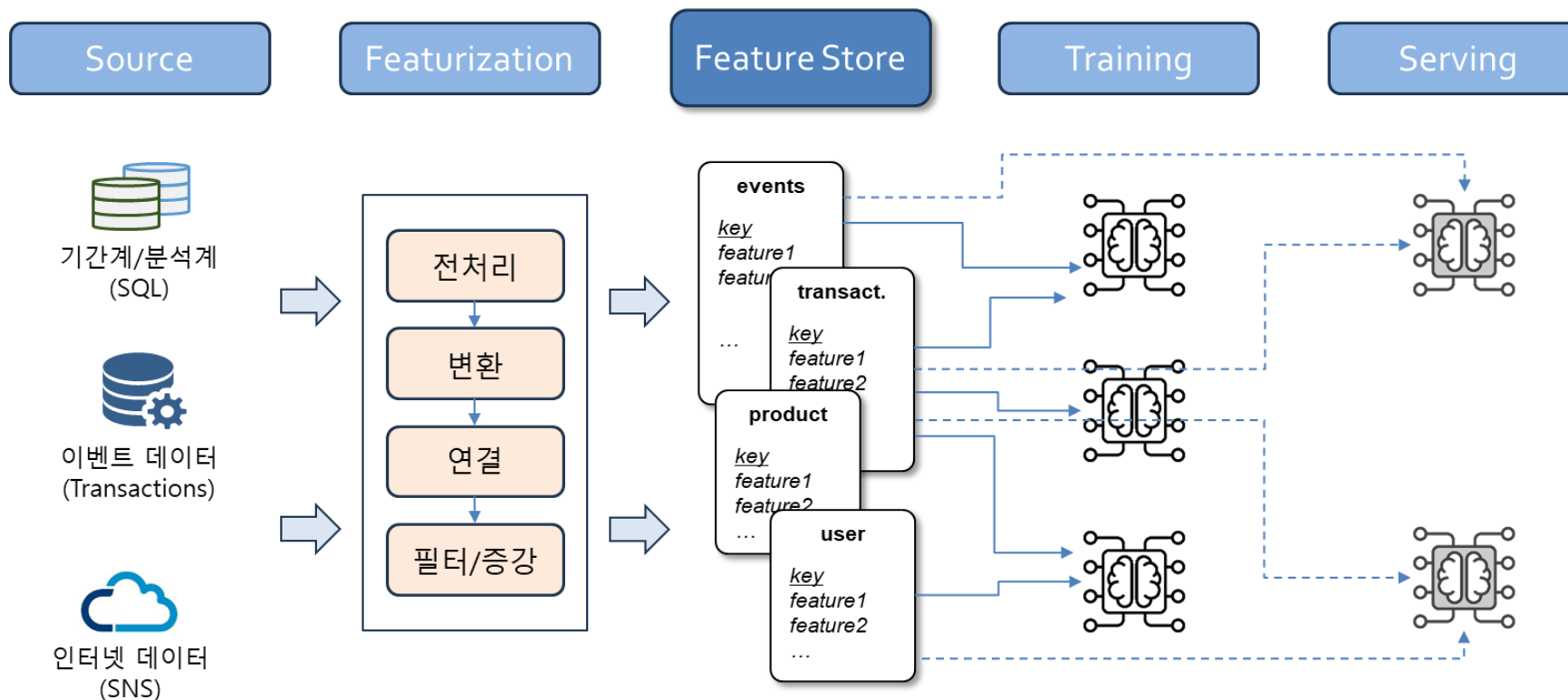


training data set

total_purch_7d	total_purch_30d	category	on_sale	purchased
25	350	clothes	Y	Y
25	350	shoes	Y	N
8	120	sports	N	Y
40	200	clothes	N	Y

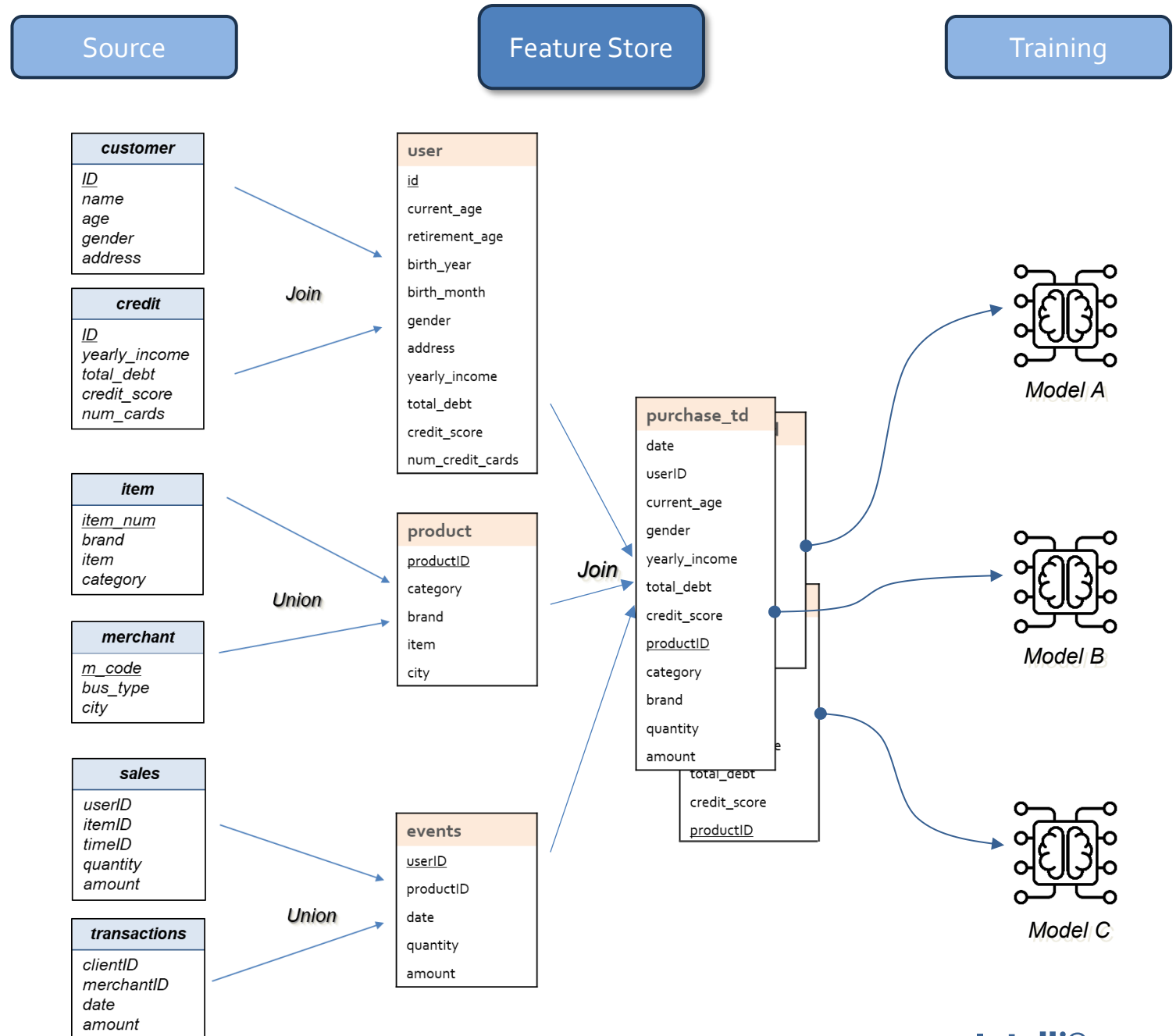
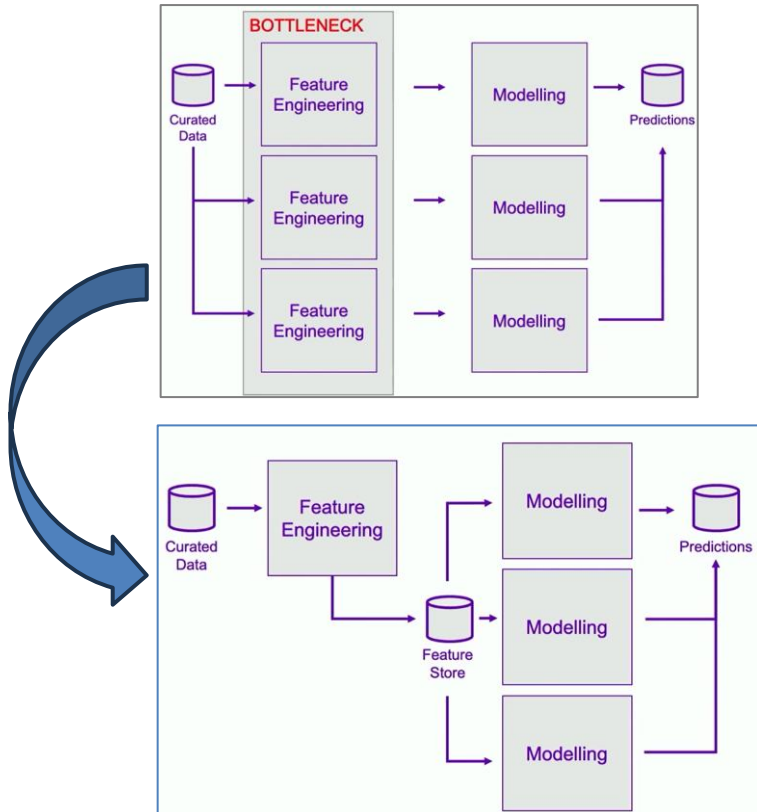
Feature Store

- Feature Store – feature들을 중앙에서 관리·공유하는 저장소
 - 학습과 추론 단계에서 feature 계산 로직의 일관성을 보장
 - 서로 다른 ML팀이 유사한 것을 중복되거나 일관성 없이 정의하는 것을 방지



Feature Store

- Feature table
 - Entity 마다 primary key와 feature들로 구성
 - 소스 데이터를 기반으로 하는 materialized view
- Feature 검색과 재사용 지원



8. LLM for DB



다양한 정보 요구

1. Einstein 교수님이 지도하는 학생들 ID
2. 교수(이름)별 지도학생 수
3. 각 연도별 학과별 개설 강좌 수, 최근연도부터 학과명 순으로
4. CS 학생들이 어느 건물에 자주 갈까
5. 상품 종류(class) 별, 상점 유형(type) 별 겨울철 연매출
6. 전체 거래 중 Visa 카드가 사용된 횟수와 비율
7. 신용점수 800 이상인 고객의 연소득 분포 그래프
8. A지점에 S카드 여신잔액 있는 고객 정보 알려줘!

생성형 AI에 대한 기대

- Text2SQL

A지점에
S카드 여신잔액
있는 고객 정보
알려줘!

AI Spreadsheets

HomeHistory로그아웃

다른 분석을 요청하고 싶으시면, "새 채팅"을 눌러주세요!

+ 새 채팅

2024년 카드 상품별 여신 잔액 보여줘

2024년 카드 상품별 여신 잔액 확인 요청

2024년 카드 상품 중 'S2 개인체크카드'의 여신 잔액이 815,026,604원으로 가장 높습니다. 다음으로 '편카드'가 611,394,596원으로 두 번째로 높은 여신 잔액을 기록했습니다.

편카드 여신잔액이 가장 높은 지점은?

2024년 편카드 여신 잔액이 가장 높은 지점 확인 요청

2024년 편카드 여신 잔액이 가장 높은 지점은 삼일로길지점이며, 여신 잔액은 132,264,430원입니다.

삼일로길지점 편카드 여신 잔액이 있는 고객 정보 알려줘

2024년 편카드 여신 잔액이 가장 높은 지점과 고객 정보 요청

삼일로길지점의 편카드 여신 잔액이 가장 높은 고객은 오호기입니다. 오호기의 여신 잔액은 94,756,609원이며, 고객번호는 5,593,475입니다.

오호기 고객님의 여신잔액이 있는 다른 상품 있어?

카드/대출

3 columns 6 rows

오호기 고객님의 여신잔액이 있는 다른 상품 있어?

삼일로길지점 편카드 여신 잔액이 있는 고객 정보 알려줘

편카드 여신잔액이 가장 높은 지점은?

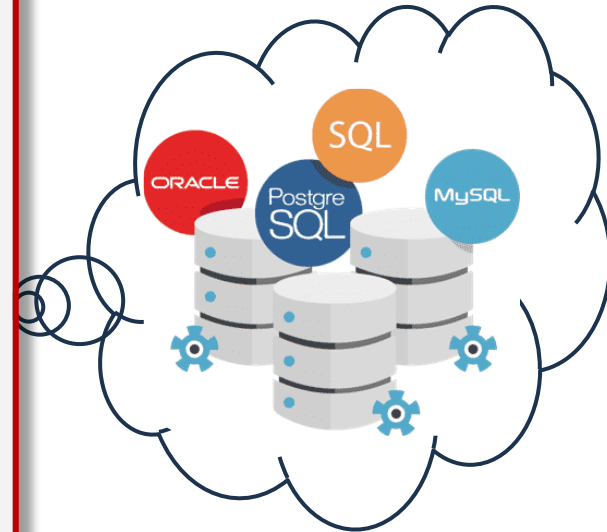
2024년 카드 상품별 여신 잔액 보여줘

전체보기

고객명	상품명	여신잔액
오호기	편카드	96,538,206
오호기	편카드	94,756,609
오호기	비대면예적금담보대출	87,227,050
오호기	심심어수메이체크카드	41,646,834
오호기	헤이(Hey)플러스대출	76,101,094
오호기	Sh모두행복계기업적금	81,936,935

« < 1 > »

Copyright © 2025 IntelliSys All Rights Reserved.



Text2SQL 어려운 이유

1. 자연어(일상 대화어)는 본질적으로 모호하고 정보가 불완전



각호텔의 평가를 알려줘

1. 각 호텔의 별점이 어떻게 되지?

```
SELECT h.name, r.stars FROM rating r JOIN hotels h  
ON h.id = r.hotel_id
```

2. 고객들이 각 호텔에 몇 점을 줬지?

```
SELECT h.name, r.guest_score FROM rating r ...
```

3. 각 호텔들의 별점과 평점을 알려줘.

```
SELECT h.name, r.stars, r.guest_score FROM rating r ...
```

rating			
id	hotel_id	stars	guest_score
1	1	★★★★	8.5

hotels	
id	name
1	Radisson

[BK Park @ SNU IDSLab]

Text2SQL 어려운 이유

2. Schema Linking – 테이블 명, 컬럼 명

cust_trx_dtl	
ACN	• Account? Action?
TRX_CD	• 거래유형? 코드? 카드?
AMT	
DT	• 거래일? 생성일?

user_info	
ID	
name	
gender	
type	• 사용자 type? 거래 type?

client_comp	
TaxID	
name	
location	• 지점? 국가?
status	• 회사 type? 거래 type?
asset	• 단위?

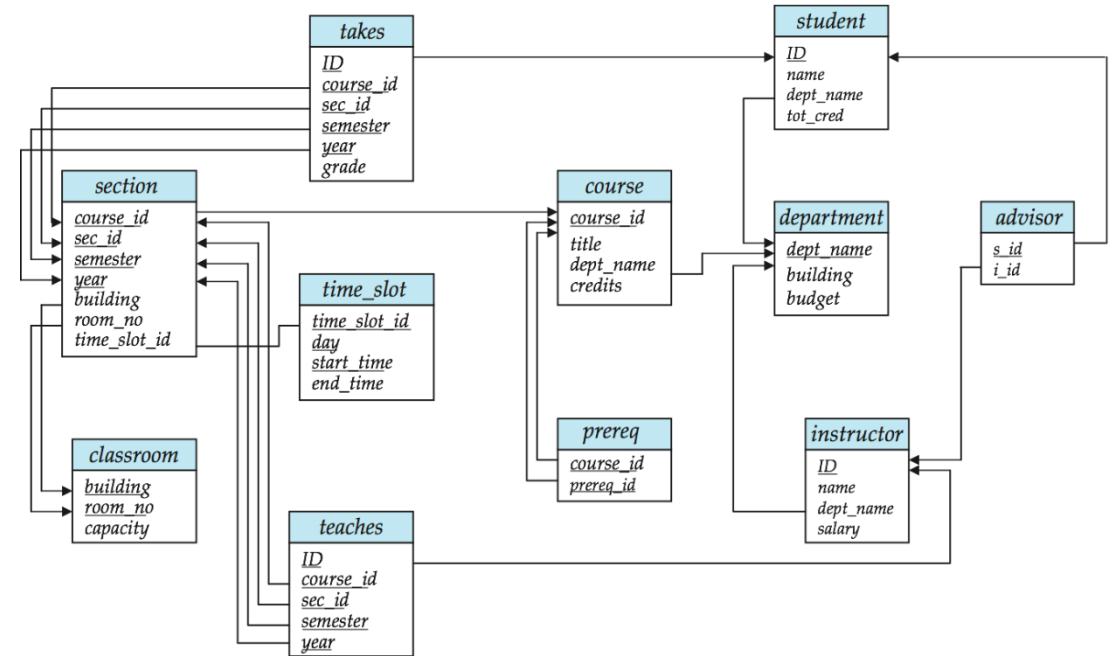
3. Schema Linking – 데이터 값 (values)

- gender: 'M', 'F', or 0, 1, or 남, 여
- '한경' 고객: *user_info.name?* *client_comp.name?* *ID?*
- 최우수 고객 이름: type? status? '최우수' or 'A'

Text2SQL 어려운 이유

4. Schema Linking – Join Path

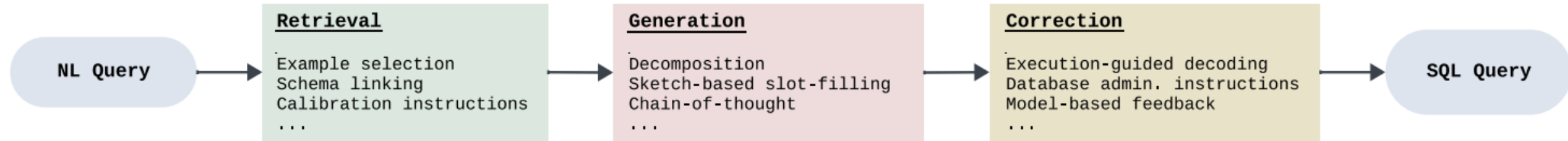
- 외래키 관계
- Einstein 교수의 수강생들
- Einstein 교수의 지도학생들
- Einstein 교수 학과 학생들



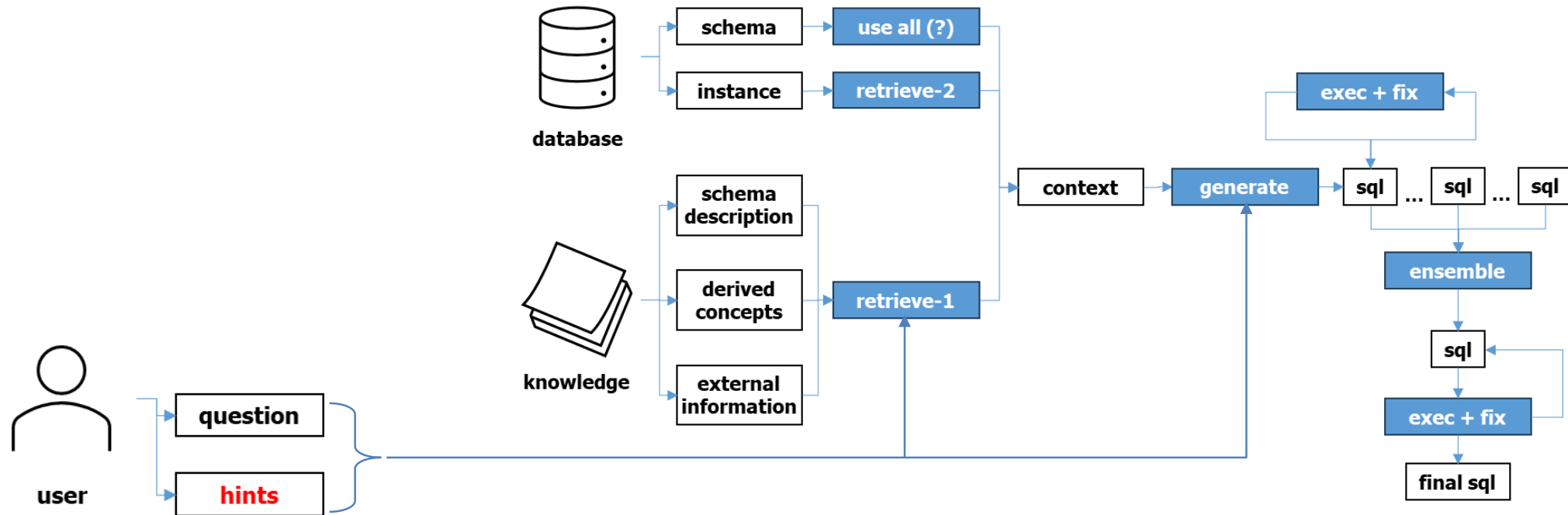
5. 전문 지식 (Domain Knowledge)

- 수율 (yield) 계산식
- 특목고: *school.type* in { '과학고', '외국어고', '예술고', '체육고' }
- 연체 또는 부실의 정의

Text2SQL Pipeline



Typical Text-to-SQL pipeline



Text2SQL Benchmark



BIRD-SQL

A Big Bench for Large-Scale Database
Grounded Text-to-SQLs

- ✓ **12,751** unique question-SQL pairs
- ✓ **95** big databases with a total size of **33.4 GB**
- ✓ covers more than **37** professional domains

Leaderboard - Execution Accuracy (EX)

	Model	Code	Size	Oracle Knowledge	Dev (%)	Test (%)
	Human Performance <i>Data Engineers + DB Students</i>			✓		92.96
1 Mar 11, 2025	AskData + GPT-4o <i>AT&T CDO - DSAIR</i>		UNK	✓	75.36	77.14
2 Apr 16, 2025	CHASE-SQL + Gemini <i>Google Cloud</i> [Pourreza et al. '24]		UNK	✓	74.90	76.02
3 Feb 27, 2025	Contextual-SQL <i>Contextual AI</i>		UNK	✓	73.50	75.63
4 Dec 17, 2024	XiYan-SQL <i>Alibaba Cloud</i> [Yifu Liu et al. '24]	[link]	UNK	✓	73.34	75.63
5 Oct 27, 2024	ExSL + granite-34b-code <i>IBM Research AI</i>		34B	✓	72.43	73.17
6 Mar 31, 2025	Reasoning-SQL 14B <i>Google Cloud / Stanford</i> [Pourreza et al. '25]		14 B	✓	72.29	72.78
7 Aug 21, 2024	OpenSearch-SQL, v2 + GPT-4o <i>Alibaba Cloud</i> [Xiangjin Xie et al. '25]	[link]	UNK	✓	69.30	72.28
8 Mar 4, 2025	OmniSQL-32B <i>Renmin University of China + ByteDance Infra Lab</i> [Li et al. '25]	[link]	32B	✓	69.23	72.05
9 Jul 22, 2024	Distillery + GPT-4o <i>Distyl AI Research</i> [Maamari et al. '24]		UNK	✓	67.21	71.83
10 Apr 18, 2025	GenaSQL <i>Gena AI</i> gena.co		UNK	✓	69.25	71.38

분석용 Text2SQL

- 주제별 view에 대해 자연어로 정보 요구
 - 엑셀 스프레드시트에 대해 정보요구 또는 조작명령 하듯이 사용

The screenshot displays the 'AI Spreadsheets' web application. The main area shows a table titled '월별 판매량 높은 상품 1개씩 | 제품별 판매량' with columns for '월' (Month), '상품코드' (Product Code), '제품명' (Product Name), and '판매수량' (Sales Volume). The table contains 4 rows of data. Above the table, there are search filters and a '기분시트' (Mood Sheet) button. To the right, the 'Ask Insight' sidebar provides a list of insights related to the selected data, such as '판매량이 높은 상품의 특징은 다음과 같습니다:' (Features of products with high sales volume are as follows:). The interface includes navigation links like 'Home', 'History', and 'logout' at the top right.

월	상품코드	제품명	판매수량
1	P0000H0F	[예약판매] 그레비티 헤어리프팅 샴푸 엑스트라 스트롱 *	418
2	P0000H0T	[단독처리가] 이질남딴 칸나물 나뭇바탕 (무정사제...	1560
3	P0000H0C	[단독처리가] 바이오세미업 NMN 안티에이징 리포좀 크...	668
4	P0000H0F	[4월 2일 출고] 그레비티 헤어리프팅 샴푸 엑스트라 스트...	510

다른 분석을 요청하고 싶으시면, "새 채팅" 을 눌러주세요!

+ 새 채팅

73 columns 500 rows

엑셀 다운로드

SQL 보기

전체 보기

TB_SFAT고객통합실적

OT_CANM계좌일별실적

TB_SOR_PDF_PRD_BC

정정보기본

고객구분코드

성별코드

고객연령구간코드

crm등급코드

고객대면비대면구분코드

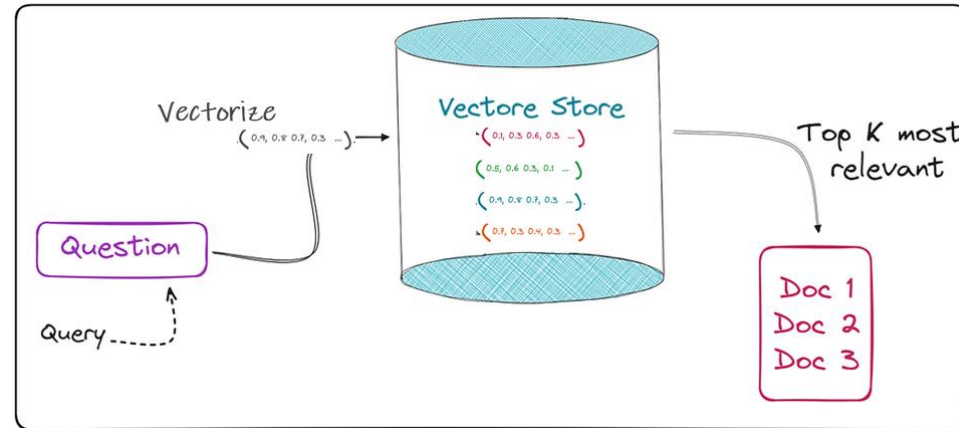
중앙회조합구분코드

기준일자	고객번호	주거래점번호	주거래점명	고객명	고객구분코드
20231231	100006259	0322	신설동역지점	자물금	01
20231231	100006359	0285	청라지점	호흥강	01
20231231	100006394	0331	인계동지점	선홍담	01
20231231	100006702	0087	공릉동금융센터	만주어	01
20231231	100006748	0227	학익동지점	만공한	01
20231231	100006775	0705	부경대금융센터	타종아	01
20231231	100006815	0201	강원지역금융본부	자아초	01
20231231	100006879	0202	춘천지점	타김팡	01
20231231	100006911	0043	수유동지점	약팡코	01
20231231	100006995	0110	바산동금융센터	자만장	01
20231231	100007001	0227	학익동지점	홍충왕	01
20231231	100007007	0124	루원시티지점	코논나	01
20231231	100007065	0105	주안지점	코송형	01
20231231	100007195	0041	대림동지점	충인말	01
20231231	100007210	0402	전주지점	충오어	01
20231231	100007365	0203	원주금융센터	충주담	01
20231231	100007467	0028	동대문금융센터	인봄유	01
20231231	100007537	0703	범일동지점	범충담	01
20231231	100007684	0701	부산지역금융본부	자다목	01
20231231	100007702	0043	수유동지점	코충박	01
20231231	100007709	0072	암사역지점	인창유	01

9. DB for LLM

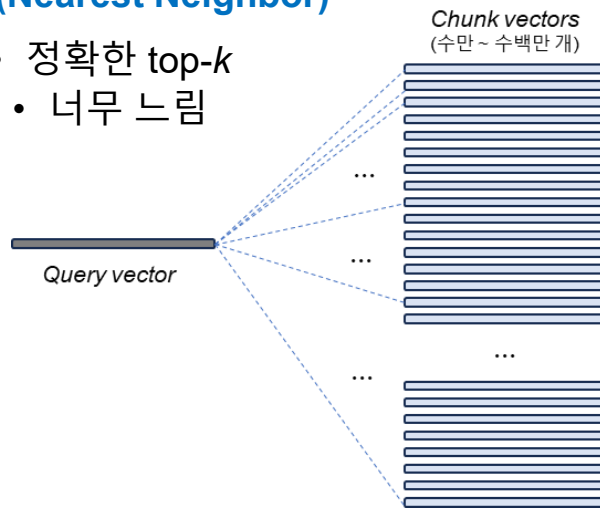


DB for LLM – Vector DB

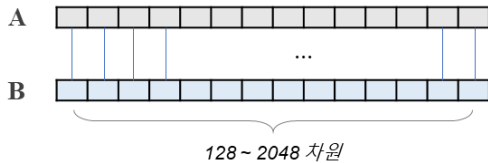


K-NN (Nearest Neighbor)

- 정확한 top-k
- 너무 느림



$$\text{similarity} = \cos(\theta) = \frac{\mathbf{A} \cdot \mathbf{B}}{\|\mathbf{A}\| \|\mathbf{B}\|} = \frac{\sum_{i=1}^n A_i B_i}{\sqrt{\sum_{i=1}^n A_i^2} \sqrt{\sum_{i=1}^n B_i^2}},$$



ANN (Approximate Nearest Neighbor)

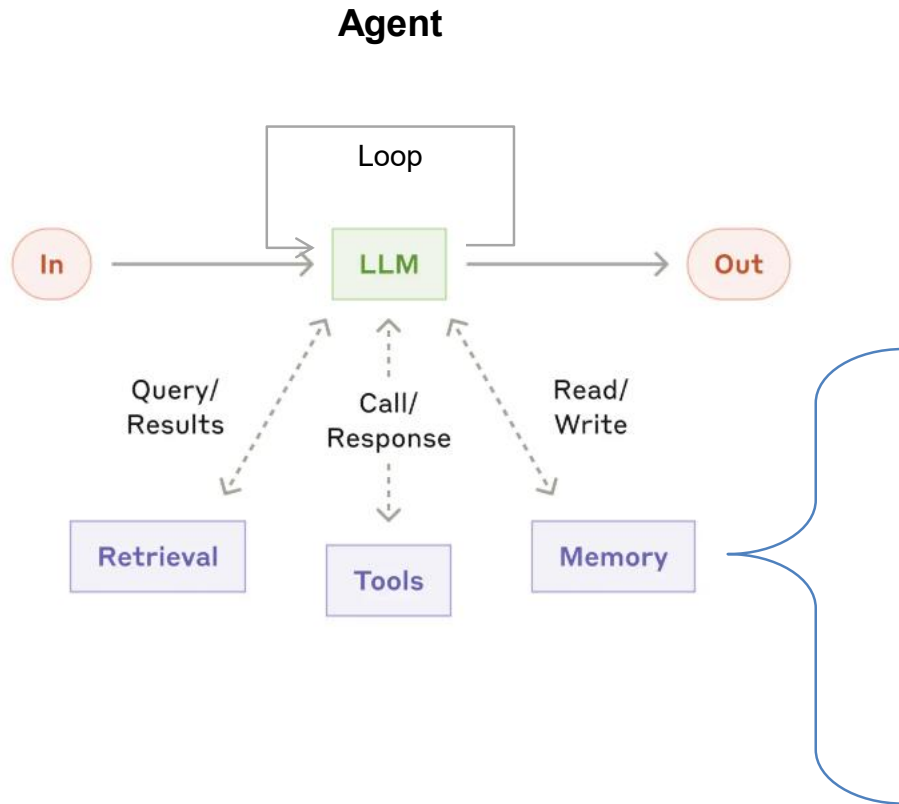
- 약간 부정확한 top-k
- 매우 빠름

+

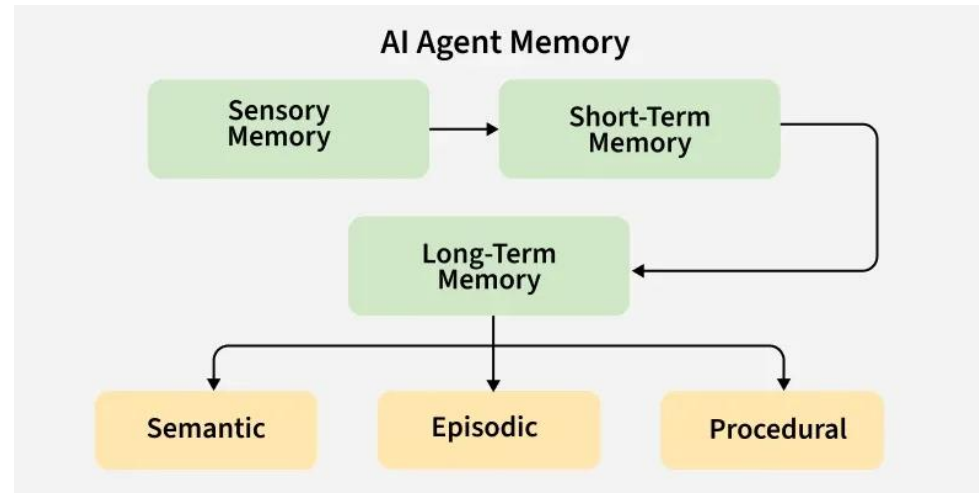
Hybrid Retrieval

- 키워드 검색과 병합
 - BM25 등
- Filter로도 사용 (meta data)

DB for Agents



- Semantic memory
 - 사실 및 상식
- Episodic memory
 - 사건과 경험
- Procedure memory
 - 태스크 처리 절차 등



우리금융그룹 AI/데이터 전문가 과정

DB for AI

Prof. Sang-goo Lee

sglee@snu.ac.kr



Intelligent Data Systems Lab.
Seoul National University

IntelliSys
A.I. DRIVEN INNOVATIONS